Labs 1-5

Student ID: 22756463

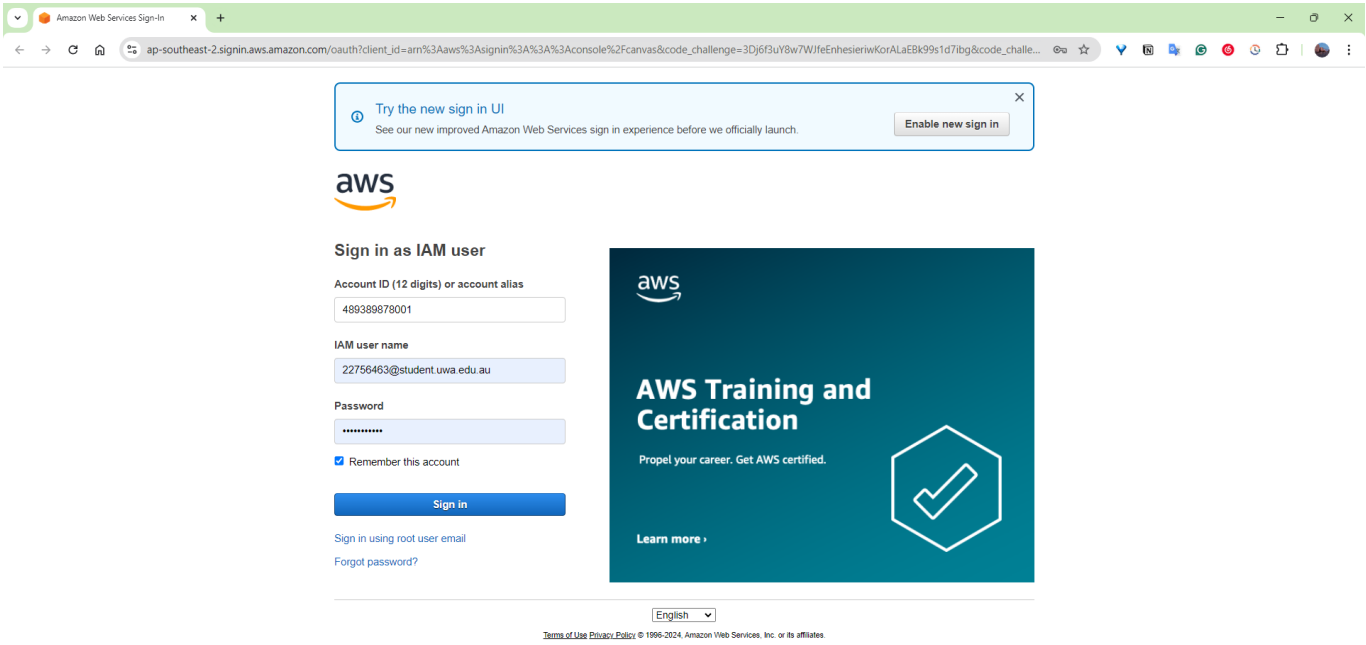
Student Name: Mila (Yue) Zhang

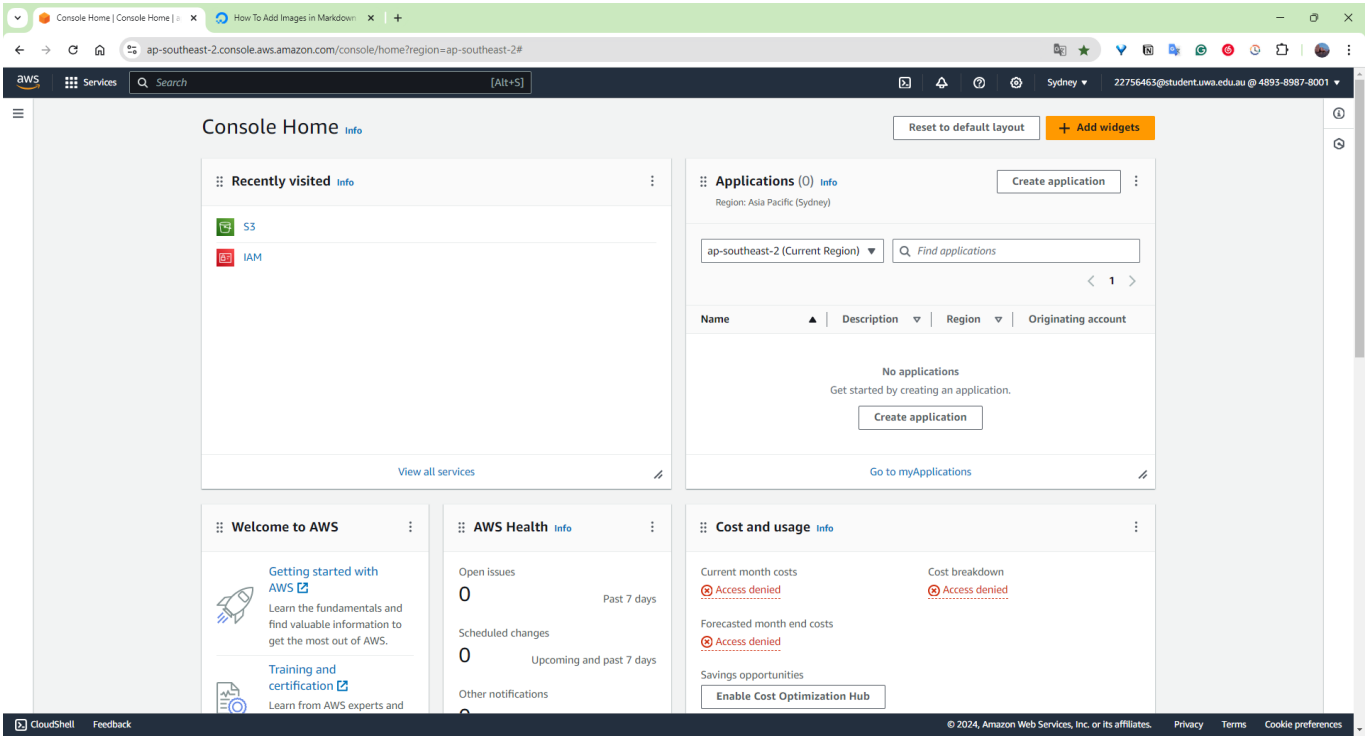
Lab 1

AWS Account and Log in

[1] Log into an IAM user account

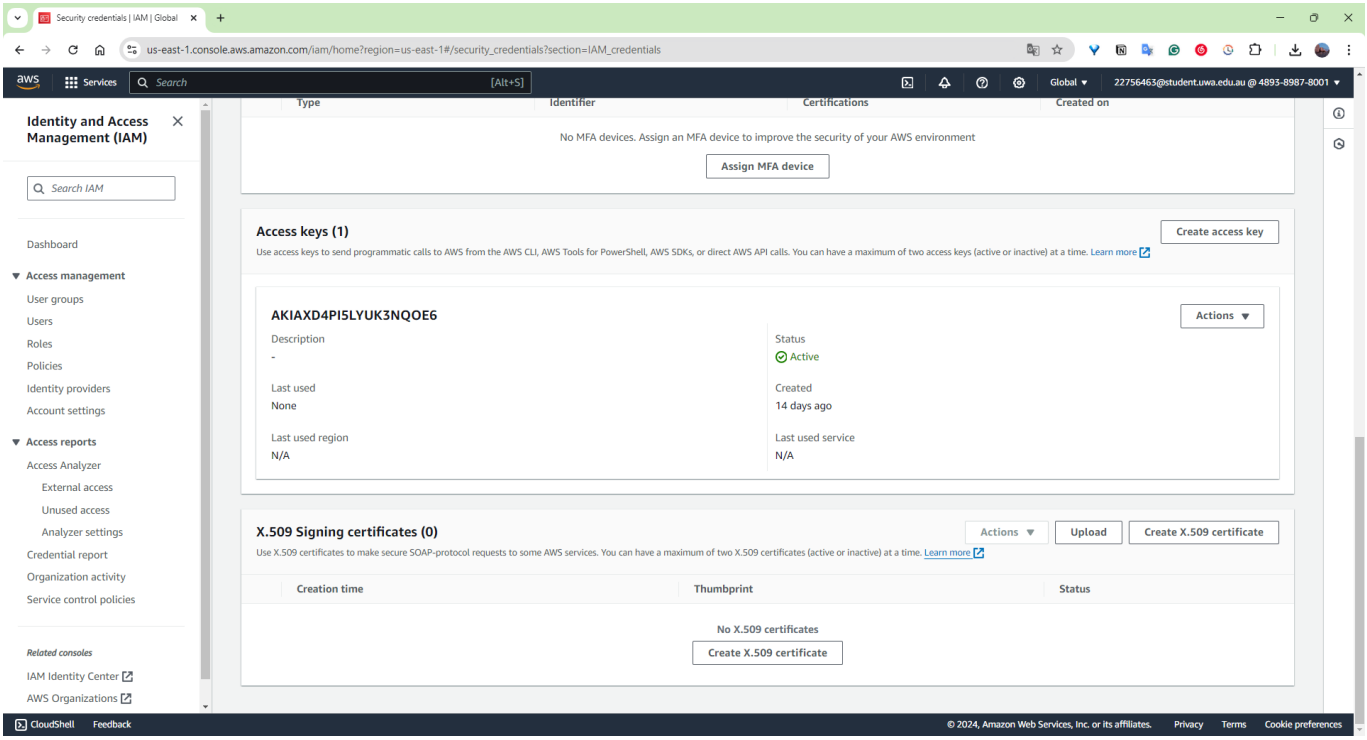
After opening the login page, I entered my IAM user name "22756463@student.uwa.edu.au" and the initial password provided by the Unit Coordinator. Then I changed my password after login.





[2] Search and open Identity Access Management

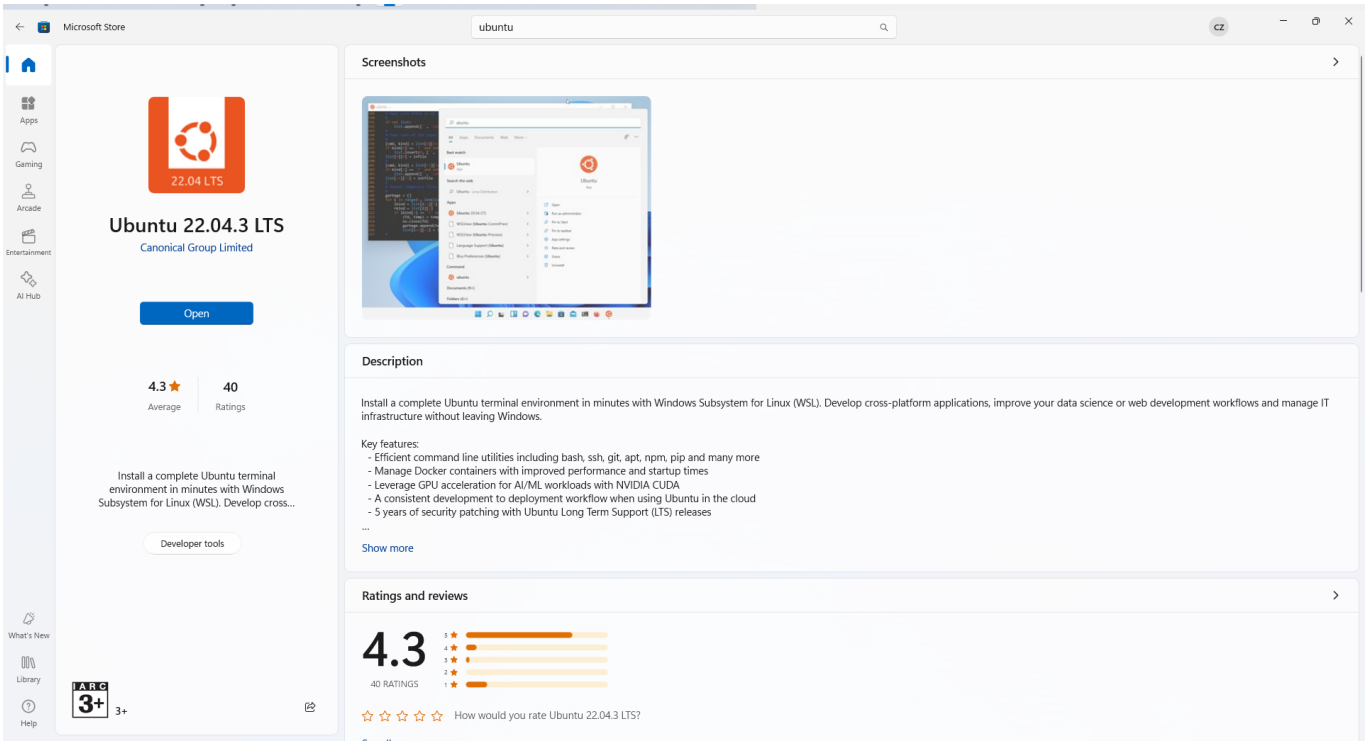
After clicking my user account and selecting the Security Credentials tab, I created an access key and noted the Access Key ID and the secret access key down. An Access Key in AWS is used to authenticate and authorise programmatic access to AWS services.



Set up recent Linux OSes

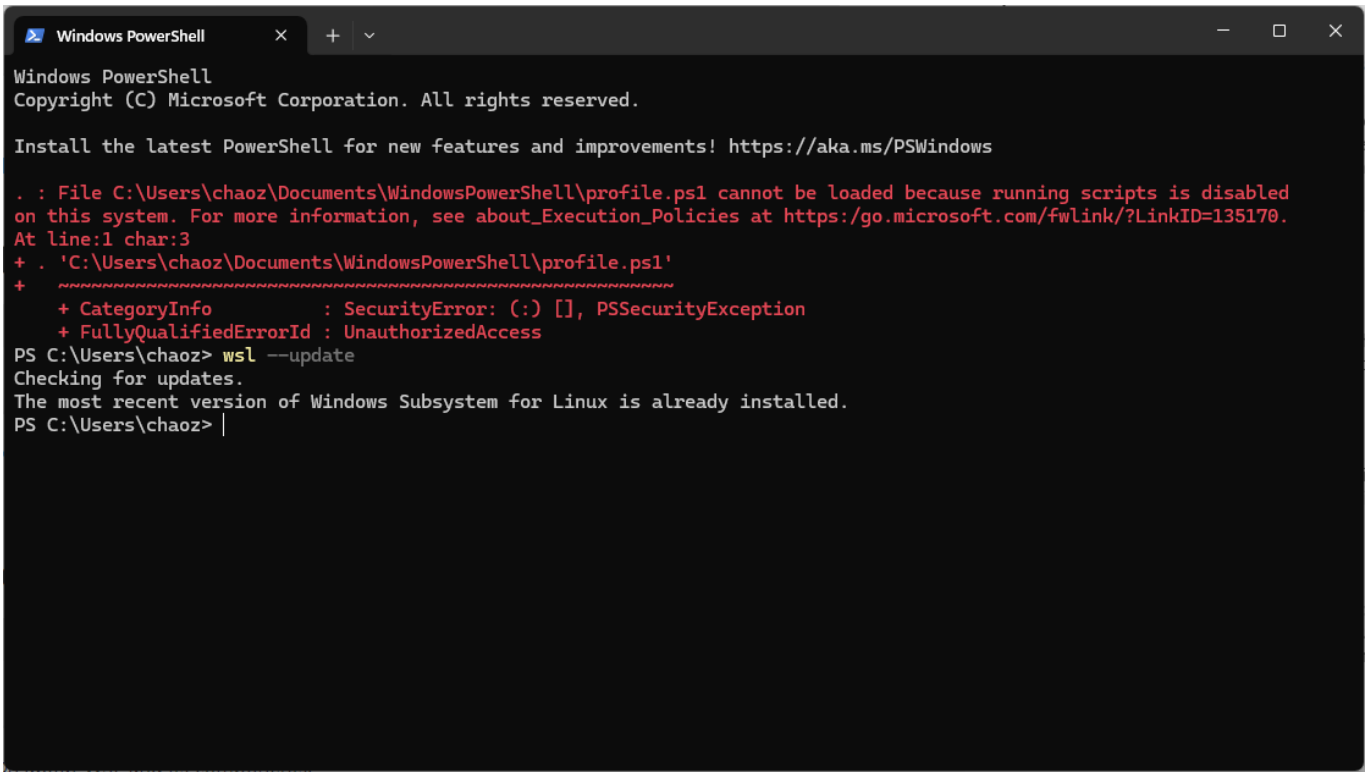
[1] Download Ubuntu OS 22.04.3

I downloaded Ubuntu OS 22.04.3 from the Microsoft Store as recommended by a lab facilitator.



[2] Update WSL

Since there was an old version of WSL installed on my laptop, I used command `wsl --update` to update the WSL.



Install Linux packages

[1] Install Python 3.8.x and above

Since the recent Ubuntu versions such as 22.04.3 LTS have Python 3.8.x installed, I updated the packages to obtain the latest version through commands `sudo apt update` and `sudo apt -y upgrade`. After checking the latest version through command `python3 -V`, now we can see I have version `Python 3.10.12` installed. Then I installed pip3 through command `sudo apt install -y python3-pip` for future Python library management.

Commands:

- `sudo`: allows us to run the command with elevated (root) privileges.
- `apt`: the command-line tool used to interact with the Advanced Package Tool (APT), the package management system used by Debian-based distributions. APT manages the installation, upgrade, and removal of software packages.
- `-y`: automatically answers "yes" to any prompts that might come up during the upgrade process.

Results:

Installed the Python version `Python 3.10.12`.

[2] Install awscli

I installed and updated awscli through commands `sudo apt install awscli` and `pip3 install awscli --upgrade`.

Commands:

- `sudo`: allows us to run the command with elevated (root) privileges.
- `apt`: the command-line tool used to interact with the Advanced Package Tool (APT), the package management system used by Debian-based distributions. APT manages the installation, upgrade, and removal of software packages.
- `pip3`: package installer for Python 3 which allows us to install and manage Python packages.
- `awscli`: It stands for Amazon Web Services Command Line Interface. It's a unified tool that provides a consistent interface for interacting with all of Amazon Web Services (AWS).

Results:

```
mila@DESKTOP-N95AEOP:~$ sudo apt install awscli
[sudo] password for mila:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
awscli is already the newest version (1.22.34-1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
--------------------------------------------------------------------------------
mila@DESKTOP-N95AEOP:~$ pip3 install awscli --upgrade
Successfully installed awscli-1.34.2 botocore-1.35.2
```

[3] Configure AWS

I set up and managed my AWS credentials and configuration settings for the AWS Command Line Interface by using command `aws configure`. I entered information of my AWS Access Key ID, AWS Secret Access Key, default region name according to my student number in the table (my student number 22756463 was allocated to region with name "us-east-1"), and the default output format.

[4] Install boto3

I installed the "boto3" library which is the official AWS SDK (software development kit) for Python through command `pip3 install boto3`.

Commands

- `pip3`: "pip" is the Python Package Installer used to install Python libraries and packages.
- `install`: The install argument tells pip3 to download and install the specified package.
- `boto3`: "boto3" allows Python developers to write software that use Amazon services like S3, EC2 and so on. After installation, we can import the library in Python scripts and interact with AWS services.

Results

Installed boto3.

Test the installed environment

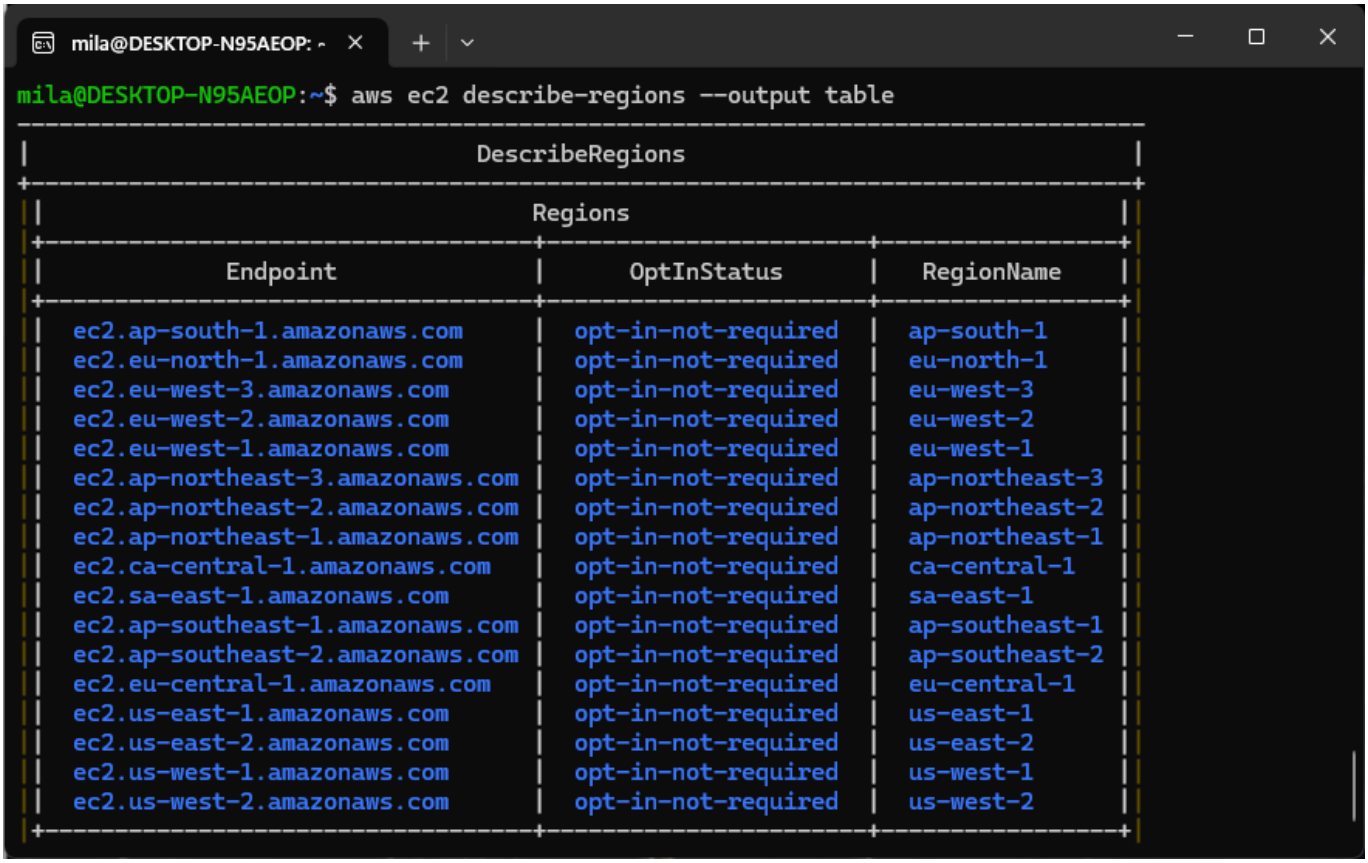
[1] Test the AWS environment

I tested the AWS environment by retrieving a list of all available AWS regions for EC2 and display the information in a table format by using command `aws ec2 describe-regions --output table`.

Commands

- `aws`: the AWS Command Line Interface. It is the tool we use to interact with AWS services directly from the command line or terminal.
- `ec2`: EC2 stands for the AWS Elastic Compute Cloud. By specifying `ec2` in this command, we tell the AWS CLI we want to perform an action associated with the EC2 service.
- `describe-regions`: the action we want to perform within the EC2 service. It lists all available AWS regions (geographical areas that have various data centres) for the EC2 service.
- `--output`: controls the format of the command's output. Some typical formats are "json", "table", "text", "yaml" and so on.
- `--output table`: this part specifies that the output should be formatted as a table.

Results



[2] Test the Python environment

I tested the Python3 environment by activating the Python3 interactive shell and entered some Python code.

Commands

```
python3
>>> import boto3
>>> ec2 = boto3.client('ec2')
```

```
>>> response = ec2.describe_regions()
>>> print(response)
```

- `import boto3`: imports the "boto3" library which is the AWS Software Development Kit for Python. After importing the package, our Python scripts can interact with AWS services.
- `ec2 = boto3.client('ec2')`: creates a client object for the EC2 service so we can make specific requests to the EC2 service.
- `response = ec2.describe_regions()`: calls the "describe_regions()" method of the EC2 client object and saves the returned information into a variable called "response".
- `print(response)`: prints out the variable "response" which is a dictionary containing information about the regions.

Results


```
{'Regions': [{'Endpoint': 'ec2.ap-south-1.amazonaws.com', 'RegionName': 'ap-south-
1', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.eu-north-
1.amazonaws.com', 'RegionName': 'eu-north-1', 'OptInStatus': 'opt-in-not-
required'}, {'Endpoint': 'ec2.eu-west-3.amazonaws.com', 'RegionName': 'eu-west-3',
'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.eu-west-2.amazonaws.com',
'RegionName': 'eu-west-2', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint':
'ec2.eu-west-1.amazonaws.com', 'RegionName': 'eu-west-1', 'OptInStatus': 'opt-in-
not-required'}, {'Endpoint': 'ec2.ap-northeast-3.amazonaws.com', 'RegionName':
'ap-northeast-3', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.ap-
northeast-2.amazonaws.com', 'RegionName': 'ap-northeast-2', 'OptInStatus': 'opt-
in-not-required'}, {'Endpoint': 'ec2.ap-northeast-1.amazonaws.com', 'RegionName':
'ap-northeast-1', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.ca-
central-1.amazonaws.com', 'RegionName': 'ca-central-1', 'OptInStatus': 'opt-in-
not-required'}, {'Endpoint': 'ec2.sa-east-1.amazonaws.com', 'RegionName': 'sa-
east-1', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.ap-southeast-
1.amazonaws.com', 'RegionName': 'ap-southeast-1', 'OptInStatus': 'opt-in-not-
required'}, {'Endpoint': 'ec2.ap-southeast-2.amazonaws.com', 'RegionName': 'ap-
southeast-2', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.eu-central-
1.amazonaws.com', 'RegionName': 'eu-central-1', 'OptInStatus': 'opt-in-not-
required'}, {'Endpoint': 'ec2.us-east-1.amazonaws.com', 'RegionName': 'us-east-1',
'OptInStatus': 'opt-in-not-required'}, {'Endpoint': 'ec2.us-east-2.amazonaws.com',
'RegionName': 'us-east-2', 'OptInStatus': 'opt-in-not-required'}, {'Endpoint':
'ec2.us-west-1.amazonaws.com', 'RegionName': 'us-west-1', 'OptInStatus': 'opt-in-
not-required'}, {'Endpoint': 'ec2.us-west-2.amazonaws.com', 'RegionName': 'us-
west-2', 'OptInStatus': 'opt-in-not-required'}], 'ResponseMetadata': {'RequestId':
'4a4ee39e-9960-4855-855f-1d1d31c7d4ed', 'HTTPStatusCode': 200, 'HTTPHeaders': {'x-
amzn-requestid': '4a4ee39e-9960-4855-855f-1d1d31c7d4ed', 'cache-control': 'no-
cache, no-store', 'strict-transport-security': 'max-age=31536000;
includeSubDomains', 'vary': 'accept-encoding', 'content-type':
'text/xml;charset=UTF-8', 'content-length': '2890', 'date': 'Wed, 21 Aug 2024
12:57:45 GMT', 'server': 'AmazonEC2'}, 'RetryAttempts': 0}}
```


[3] Write a Python script

To tabulate the above returned response, I used the Python code below.

Commands

- `pip3 install tabulate`: Since the package "tabulate" was not installed before, I install the library first, so we can easily format tabular data by using functions of this library.

```
import boto3
from tabulate import tabulate

# Create an EC2 client object
ec2 = boto3.client('ec2')

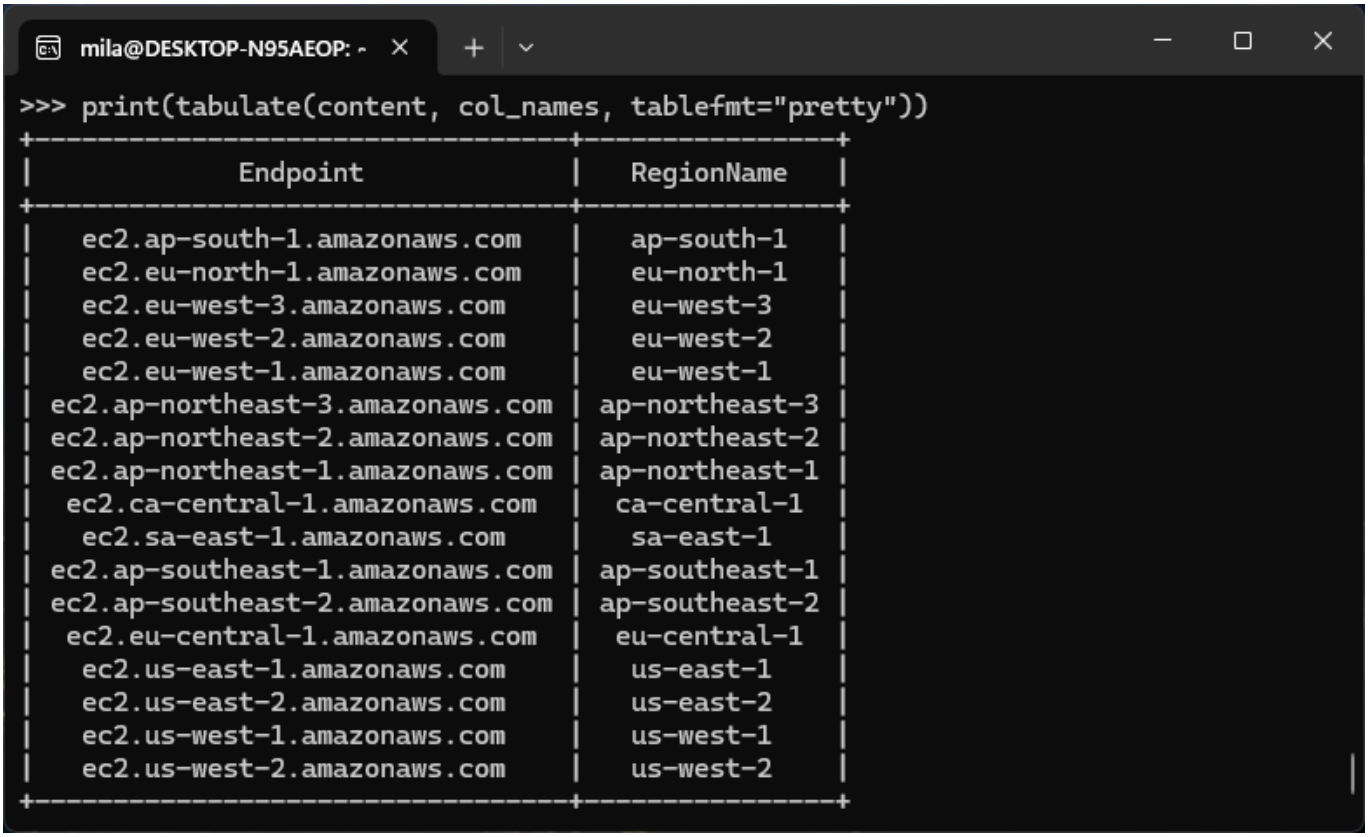
# Call the describe_regions method and store in the variable response
response = ec2.describe_regions()

# This list comprehension iterates over each dictionary in "response['Regions']".
Then it extracts the Endpoint and RegionName from each dictionary and stores them
as a list of tuples
content = [(region['Endpoint'], region['RegionName']) for region in
response['Regions']]

# Define the column names for the table
col_names = ["Endpoint", "RegionName"]

# Print the table using tabulate
print(tabulate(content, col_names, tablefmt="pretty"))
```

Results



Lab 2

Create an EC2 instance using awscli

[1] Create a security group

I created a new security group in Amazon EC2 through the command `aws ec2 create-security-group --group-name 22756463-sg --description "security group for development environment"`. This security group will act as virtual firewalls to control inbound and outbound traffic (traffic coming into the instances and traffic going out from the instances) to the EC2 instances.

Commands

- `aws ec2`: refers to the AWS EC2 service. By specifying this, we are using the AWS EC2 service through the command line.
- `create-security-group`: the action we are doing given this command line. It creates a new security group in the default VPC (Virtual Private Cloud) given VPC is not specified in this case.
- `--group-name`: this flag is used to specify the name of the security group.
- `-sg`: "sg" stands for security group. This suffix is a common convention to specify that this is a security group.
- `--description`: this flag is used to set the description for the security group, so we can identify the purpose of this security group later.

Results

Created a security group named "22756463-sg".

```
{
    "GroupId": "sg-0ae5c80f51af3b210"
}
```

[2] Authorise inbound traffic for ssh

I authorised inbound traffic by using command `aws ec2 authorize-security-group-ingress --group-name 22756463-sg --protocol tcp --port 22 --cidr 0.0.0.0/0`. This command allowed inbound SSH (port 22) traffic to reach my EC2 instance from any IP address (by specifying "0.0.0.0/0"), as long as the instance is associated with the security group "22756463-sg". This rule is applied using the TCP protocol.

Commands

- `aws ec2 authorize-security-group-ingress`: This is used to add a rule to an existing security group to allow inbound traffic to the security group.
- `--group-name 22756463-sg`: Specifies the name of the security group we are modifying.
- `--protocol tcp`: Defines the protocol that is allowed. In this case, tcp is the Transmission Control Protocol, which is the protocol used for reliable communication over the internet (commonly used for

SSH, HTTP, HTTPS, etc.).

- `--port 22`: `--port` Specifies the port number to allow inbound traffic on. In this case, port 22 is used for SSH (Secure Shell), which is commonly used for remote access to Linux servers.
- `--cidr 0.0.0.0/0`: The CIDR (Classless Inter-Domain Routing) block represents the range of IP addresses that are allowed to connect. "0.0.0.0/0" represents all IP addresses, meaning this rule will allow SSH traffic from anywhere in the world. (This is generally not secure and should be restricted to a specific IP range or address whenever possible.)
- SSH (Secure Shell): A *network protocol* used to securely *access and manage remote servers* over an unsecured network. It allows us to establish a secure, encrypted connection between our local machine and a remote server, enabling us to execute commands, transfer files, and perform administrative tasks on that server.

Results


```
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-093d071b1a0fd2236",
            "GroupId": "sg-0ae5c80f51af3b210",
            "GroupOwnerId": "489389878001",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 22,
            "ToPort": 22,
            "CidrIpv4": "0.0.0.0/0"
        }
    ]
}
```


[3] Create a key pair

I generated a new key pair, extracted the private key and saved it to a file in the file format ".pem" which is mainly used for SSH access to EC2 instances. This private key file will be required when I SSH into my EC2 instance. I used the command `aws ec2 create-key-pair --key-name 22756463-key --query 'KeyMaterial' --output text > 22756463-key.pem`. Then I change the file permission by using command `chmod 400 22756463-key.pem`.

Commands

- `aws ec2 create-key-pair`: Used to create an SSH key pair for securely connecting to an Amazon EC2 instance. A key pair consists of a public key (stored by AWS) and a private key (saved by me), which are used to securely connect to an EC2 instance via SSH.
- `--key-name`: Specifies the name of the key pair. The key name must be unique within my AWS account for that region.
- `--query 'KeyMaterial'`: The `--query` option filters the output of the command. 'KeyMaterial' is the part of the *returned data* that contains the actual *private key* in plain text format, which I need to

connect to EC2 instances via SSH. This ensures that only the private key (KeyMaterial) is retrieved from the full response that AWS returns.
- `--output text`: This option specifies that the output should be in plain text format, rather than JSON or any other format. This ensures that the private key is displayed in a simple, readable text format, which can then be saved to a file.
- `> 22756463-key.pem`: The `>` operator redirects the output of the command to a file named `22756463-key.pem`. This file will contain the private key (in .pem format), which I will later use to connect to my EC2 instance via SSH.
- ".pem" file: A ".pem" file (Privacy Enhanced Mail) is a file format that typically contains a private key and/or a public key. It is used in various security and encryption contexts, most commonly for establishing secure communication over the internet (such as SSH and HTTPS). In AWS, .pem files are primarily used for SSH access to EC2 instances.
- `chmod 400 22756463-key.pem`: This command sets the file permissions such that only the file owner can read the 22756463-key.pem file, and no one else (group or others) can read, write, or execute the file. `chmod` is the change mode command in Unix/Linux systems. It is used to change the permissions of a file or directory. Permissions control who can read, write, or execute a file. `400` is the permission code that specifies what permissions are being applied to the file. The first digit represents the owner permission, the second digit represents the group permission, and the last digit represent the permission of others. "4" represents read permission while "0" means no permission. `22756463-key.pem` specifies the name of the file for which the permissions are being changed.

Results

Generated a new key pair, extracted the private key and saved it into "22756463-key.pem". Then changed the file permission to be read only by the owner.

[4] Create the instance

I created an instance located in the region "US East (N. Virginia)" according to my student number and the allocation table. I used the command `aws ec2 run-instances --image-id ami-0a0e5d9c7acc336f1 --security-group-ids 22756463-sg --count 1 --instance-type t2.micro --key-name 22756463-key --query 'Instances[0].InstanceId'`.

Commands

- `aws ec2 run-instances`: This command is used to launch (run) one or more EC2 instances in the AWS cloud.
- `--image-id`: Specifies the Amazon Machine Image (AMI) that will be used to launch the instance.
- `--security-group-ids`: This specifies the security group(s) that will be associated with the instance. The security group acts as a *virtual firewall* that controls inbound and outbound traffic.
- `--count`: Specifies the number of EC2 instances to launch.
- `--instance-type`: Specifies the type of EC2 instance to launch. `t2.micro` is a small, general-purpose instance type with 1 vCPU and 1 GB of RAM. It is suitable for low-traffic applications or testing and is often eligible for the AWS Free Tier.
- `--key-name`: Specifies the name of the SSH key pair that will be used to access the instance.
- `--query 'Instances[0].InstanceId'`: `--query` is used to filter the output of the command. This queries the result to return only the Instance ID of the first launched instance.

Results

The Instance ID of the newly launched instance was returned:

```
"i-04c3576556495a2d1"
```

[5] Add a tag to your Instance

I added a tag to my EC2 instance created in step 4 by using the command `aws ec2 create-tags --resources i-04c3576556495a2d1 --tags Key=Name,Value=22756463`. The tag is useful for identifying and organising the instance.

Commands

- `aws ec2 create-tags`: This command is used to assign metadata, called tags, to an AWS resource such as an EC2 instance, volume, or other AWS resources. Tags are *key-value pairs* that allow me to organise, identify, and manage my resources more easily.
- `--resources`: Specifies which AWS resources (like EC2 instances, volumes, etc.) I want to tag.
- `--tags Key=Name,Value=22756463`: `--tags` defines the tag(s) I want to apply to the resource. `Key=Name` specifies the key for the tag. In AWS, "Name" is commonly used as a tag key to give the resource a human-readable name. `Value=22756463` specifies the value for the "Name" tag.

Results

Applied a tag to my EC2 instance with the ID "i-04c3576556495a2d1". The tag has a key of "Name" and a value of "22756463".

[6] Get the public IP address

I retrieved the public IP address of the EC2 instance I created in step 4 by using command `aws ec2 describe-instances --instance-ids i-04c3576556495a2d1 --query 'Reservations[0].Instances[0].PublicIpAddress'`.

Commands

- `aws ec2 describe-instances`: The command is used to retrieve details about specific EC2 instances.
- `--instance-ids`: This flag specifies the ID(s) of the EC2 instances I want to describe.
- `--query 'Reservations[0].Instances[0].PublicIpAddress'`: `--query` filters the output to retrieve only the information I need. This query navigates the JSON structure returned by the `describe-instances` command to extract the public IP address of the first instance in the first reservation.

Results

Public IP address of the specified EC2 instance was returned:

```
"54.175.111.6"
```

[7] Connect to the instance via ssh

I connected to an Ubuntu EC2 instance located at public address "54.175.111.6" using SSH by the command `ssh -i 22756463-key.pem ubuntu@54.175.111.6`.

Commands

- `ssh`: This is the command for Secure Shell (SSH), a protocol used to securely connect to remote servers. SSH enables encrypted communication between the client (my machine) and the server (in this case, an EC2 instance).
- `-i 22756463-key.pem`: `-i` specifies the identity file. In this case is the private key that I use to authenticate with the remote server.
- `ubuntu`: This is the AMI (Amazon Machine Image) *username* that I am using to log into the remote server.
- `@54.175.111.6`: `@` separates the username (ubuntu) from the server's IP address or hostname. `54.175.111.6` is the public IP address of the EC2 instance I am connecting to.

Results

```
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.5.0-1022-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

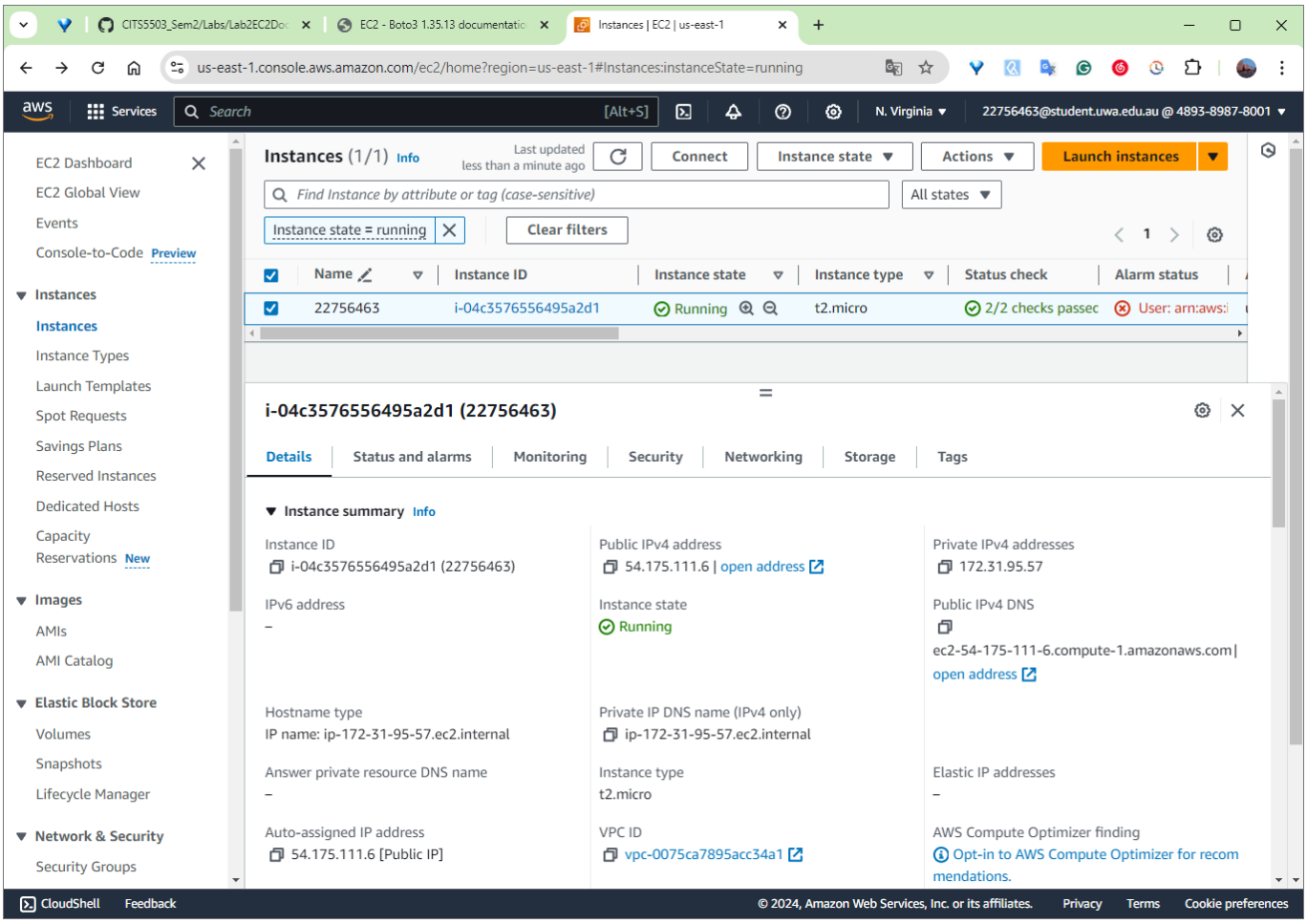
 System information as of Fri Sep  6 13:40:19 UTC 2024

  System load:  0.08              Processes:             97
  Usage of /:   20.7% of 7.57GB   Users logged in:       0
  Memory usage: 21%               IPv4 address for eth0: 172.31.95.57
  Swap usage:   0%
```

[8] List the created instance using the AWS console

I logged into AWS Console and navigated to the EC2 Dashboard to view the information about the instance I created.

Results



Create an EC2 instance with Python Boto3

I wrote the following Python script into a file in my current directory in the Ubuntu system. And then run the Python script to create an EC2 instance.

Python Script

```
import boto3
from botocore.exceptions import ClientError

# Initialize EC2 client to interact with AWS EC2 service
ec2 = boto3.client('ec2')

# Step 1: Create a security group
def create_security_group(group_name, description):
    try:
        # Create a new security group with the specified group name and
description
        # Security group acts as a virtual firewall to control inbound and
outbound traffic
        response = ec2.create_security_group(
            GroupName=group_name,
            Description=description,
        )
        security_group_id = response['GroupId']  # Get the ID of the newly created
security group
```

```
        print(f'Security Group {group_name} created with ID: {security_group_id}')
        return security_group_id
    except ClientError as e:
        # Handle any errors during security group creation
        print(f"Error creating security group: {e}")
        return None

# Step 2: Authorize inbound traffic for SSH (port 22)
def authorize_security_group_ingress(group_name):
    try:
        # Authorize SSH traffic (port 22) from any IP address (0.0.0.0/0) to the
security group
        ec2.authorize_security_group_ingress(
            GroupName= group_name,
            IpPermissions=[
                {
                    'IpProtocol': 'tcp',  # Use the TCP protocol
                    'FromPort': 22,       # Allow traffic on port 22 (SSH)
                    'ToPort': 22,
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]  # Allow access from any
IP address
                }
            ]
        )
        print(f"SSH rule added to Security Group: {group_name}")
    except ClientError as e:
        # Handle any errors during authorization
        print(f"Error adding SSH rule to security group: {e}")

# Step 3: Create a key pair for SSH access
def create_key_pair(key_name):
    try:
        # Create a new key pair with the given name for secure SSH access to the
EC2 instance
        key_pair = ec2.create_key_pair(KeyName=key_name)
        private_key = key_pair['KeyMaterial']  # Get the private key

        # Save the private key to a .pem file (used for SSH access)
        with open(f"{key_name}.pem", "w") as key_file:
            key_file.write(private_key)

        print(f"Key pair {key_name} created and saved as {key_name}.pem")
    except ClientError as e:
        # Handle any errors during key pair creation
        print(f"Error creating key pair: {e}")

# Step 4: Create an EC2 instance
def create_instance(key_name, security_group_name, ami_id):
    try:
        # Launch a new EC2 instance using the specified AMI ID, key pair, and
security group
        instances = ec2.run_instances(
            ImageId= ami_id,             # AMI ID for the instance
            MinCount=1,                  # Create one instance
```

```
            MaxCount=1,
            InstanceType='t2.micro',     # Instance type
            KeyName=key_name,            # The key pair to use for SSH access
            SecurityGroups=[security_group_name],  # Attach security group by name
        )
        instance_id = instances['Instances'][0]['InstanceId']  # Get the instance
ID
        print(f"EC2 instance created with Instance ID: {instance_id}")
        return instance_id
    except ClientError as e:
        # Handle any errors during EC2 instance creation
        print(f"Error creating EC2 instance: {e}")
        return None

# Step 5: Add a tag (name or metadata) to the EC2 instance
def add_tag(instance_id, tag_key, tag_value):
    try:
        # Add a tag to the EC2 instance for identification purposes
        ec2.create_tags(
            Resources=[instance_id],  # Target the instance by ID
            Tags=[
                {'Key': tag_key, 'Value': tag_value}  # Assign a tag key and value
(e.g., Name: instance_name)
            ]
        )
        print(f"Tag '{tag_key}: {tag_value}' added to instance {instance_id}")
    except ClientError as e:
        # Handle any errors during tagging
        print(f"Error tagging instance: {e}")

# Step 6: Get the public IP address of the EC2 instance
def get_public_ip(instance_id):
    try:
        # Retrieve information about the instance, including its public IP address
        reservations = ec2.describe_instances(InstanceIds=[instance_id])
        public_ip = reservations['Reservations'][0]['Instances'][0]
['PublicIpAddress']  # Extract the public IP
        print(f"Public IP address of the instance: {public_ip}")
        return public_ip
    except ClientError as e:
        # Handle any errors when retrieving instance details
        print(f"Error retrieving public IP: {e}")
        return None

# Main function to orchestrate the steps for creating and configuring the EC2
instance
def main():
    sg_name = "22756463-sg3"  # Name of the security group
    sg_description = "security group for development environment"  # Description
of the security group
    key_name = "22756463-key3"  # Name of the key pair
    tag_key = "Name"  # Tag key (e.g., Name)
    tag_value = "22756463-3"  # Tag value (instance name)
    ami_id = "ami-0a0e5d9c7acc336f1"  # AMI ID for the instance (Amazon Linux 2)
```

```
    # Step 1: Create a security group
    security_group_id = create_security_group(sg_name, sg_description)

    if security_group_id:
        # Step 2: Authorize inbound SSH traffic to the security group
        authorize_security_group_ingress(sg_name)

        # Step 3: Create a key pair for SSH access
        create_key_pair(key_name)

        # Step 4: Create an EC2 instance
        instance_id = create_instance(key_name, sg_name, ami_id)

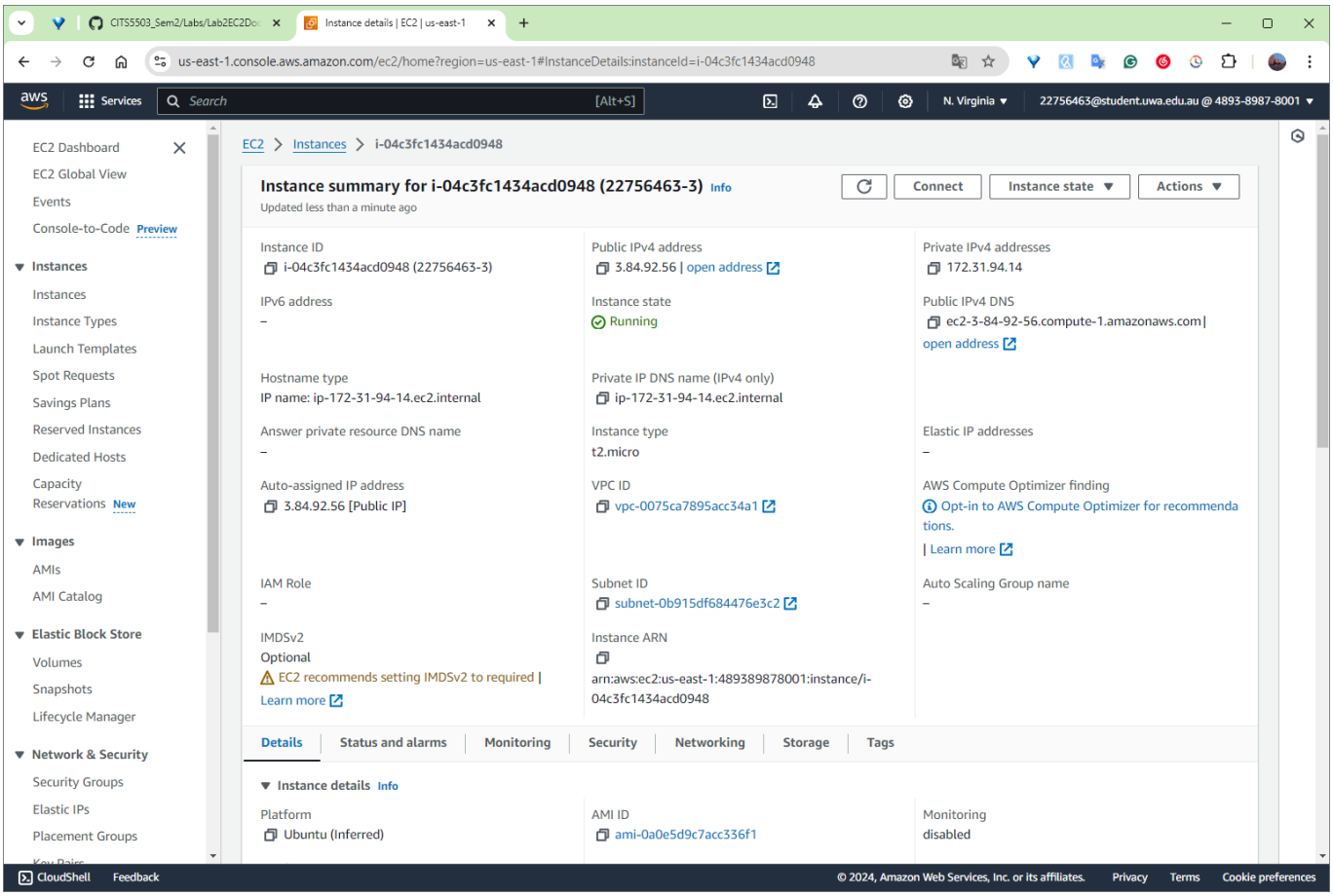
        if instance_id:
            # Step 5: Add a tag to the EC2 instance
            add_tag(instance_id, tag_key, tag_value)

            # Step 6: Get the public IP address of the instance
            get_public_ip(instance_id)

# Entry point of the script
if __name__ == "__main__":
    main()  # Call the main function to execute the process
```

A detailed breakdown is provided as comments in the code block above.

Results



Use Docker inside a Linux OS

[1] Install Docker

I installed the Docker Engine (docker.io) on the Ubuntu system by using command `sudo apt install docker.io -y`.

Commands

- `sudo`: Run the command with elevated (root) privileges.
- `apt install docker.io`: Use the "apt" package manager to install a package names "docker.io".
- `-y`: This option automatically answers "yes" to any prompts that might come up during the installation process.

Results

The Docker package was installed.

[2] Start Docker

I started the Docker service on a Linux system that uses systemd as its init system by the command `sudo systemctl start docker`.

Commands

- `sudo`: Run the command with elevated (root) privileges.

- `systemctl`: Used to control system services, including starting, stopping, restarting, enabling, and disabling services.
- `start`: This is the subcommand that tells systemctl to start the specified service. In this case, we are starting the Docker service.

Results

The Docker service was started.

[3] Enable Docker

I configured my system so that the Docker service automatically starts every time the system boots by using command `sudo systemctl enable docker`.

Commands

- `sudo`: Run the command with elevated (root) privileges.
- `systemctl`: The command-line utility used to interact with `systemd`, the service manager on many Linux distributions.`systemctl` is used to manage services, including enabling, disabling, starting, and stopping services.
- `enable`: This subcommand tells systemctl to enable the specified service. In this case, enable Docker.

Results

Enabled the Docker service to start at boot.

[4] Check the version

I checked the Docker package version by command `docker --version`.

Results

The Docker version information is returned:

```
Docker version 24.0.7, build 24.0.7-0ubuntu2~22.04.1
```

[5] Build and run an httpd container

1. Prepare HTML file and the Dockerfile

I created a directory called html by using command `mkdir html`. Then I navigated to the "html" directory by using command `cd html`. I then used command `nano index.html` and copied the html content into the the file. After saving the file, I changed back to the parent directory by using command `cd ..`. Then I created a file called Dockerfile by using command `nano Dockerfile`.

2. Build a docker image

I built a Docker image from a Dockerfile in the current directory and tagged it with a name by using command `sudo docker build -t my-apache2 .`.

Commands

- `sudo`: Run the command with elevated (root) privileges.
- `docker build`: This command tells Docker to build an image. It reads the instructions in the Dockerfile to build the image.
- `-t my-apache2`: The `-t` option stands for "tag". It allows me to name the image with a custom tag. `my-apache2` is the name (or tag) we are assigning to the image. This makes it easier to reference the image later when running or managing containers.
- `.`: Refers to the current directory.

Results

```
Sending build context to Docker daemon  77.13MB
Step 1/2 : FROM httpd:2.4
2.4: Pulling from library/httpd
a2318d6c47ec: Pull complete
62dd86107c65: Pull complete
4f4fb700ef54: Pull complete
22871f73faed: Pull complete
ca061a523d1f: Pull complete
509789394c2a: Pull complete
Digest: sha256:ae1124b8d23ee3fc35d49da35d5c748a2fce318d1f55ce59ccab889d612f8be8
Status: Downloaded newer image for httpd:2.4
 ---> 9cb0a2315602
Step 2/2 : COPY ./html/ /usr/local/apache2/htdocs/
 ---> 6f9ac4719994
Successfully built 6f9ac4719994
Successfully tagged my-apache2:latest
```

3. Run the image

I run the Docker image by using command `sudo docker run -p 80:80 -dit --name my-app my-apache2`. And then open it in the browser through address "http://127.0.0.1".

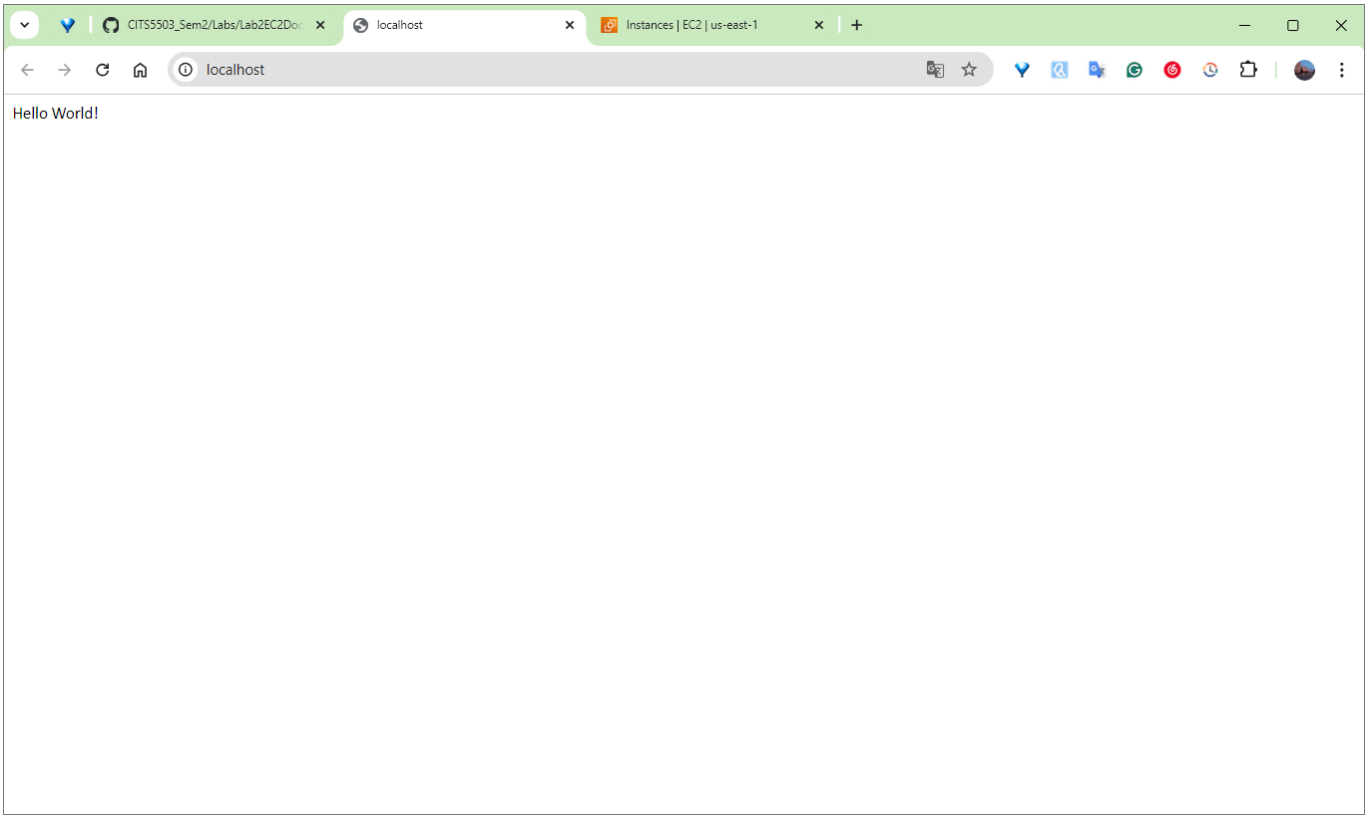
Commands

- docker run: The command is used to run a container from a Docker image. It creates a new container, starts it, and runs the application or service defined in the image.
- `-p 80:80`: The -p option maps a port from the host to the container. In this case, it maps port 80 of the host to port 80of the container. This allows the container to serve web traffic via the host's port 80.
- `-dit`: This is a combination of three flags:
  - `-d`: Runs the container in detached mode, meaning it runs in the background and doesn't block your terminal. The container continues running until you stop it.

- `-i`: Runs the container in interactive mode, allowing you to interact with the container. This is useful if you want to keep STDIN open for input (though combined with -d, it will allow the container to keep running).
- `-t`: Allocates a pseudo-TTY (a terminal interface), which allows you to interact with the container through a terminal if needed. This is useful for containers that require terminal interaction.

Results

```
4bd9b9e14d0e26f1b725190c3a2589130fe43359f9b47b7b061c72ef678c5f62
```



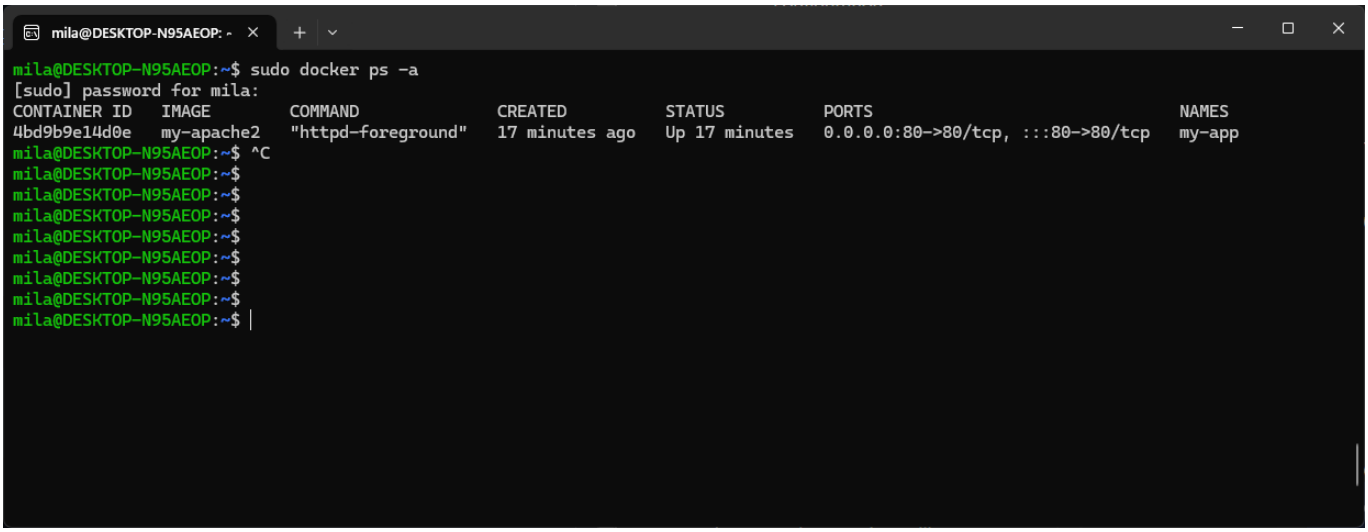
[6] Other docker commands

Check what is running

Commands

- `ps`: This is the "process status" command in Docker. It shows information about Docker containers. Without any options, docker `ps` will only show running containers.
- `-a`: The -a (or --all) option modifies the docker ps command to show all containers.

Results



Stop and remove the container

I stopped and removed the container by using commands `sudo docker stop my-app` and `sudo docker rm my-app`.

Lab 3

[1] Preparation

I prepared items for the program as follows:

1. I downloaded the Python code `cloudstorage.py` from the directory of src.
2. I created a directory called `rootdir` by using command `mkdir rootdir`.
3. Then I changed into the directory `rootdir` by using command `cd rootdir` and write content "1\n2\n3\n4\n5\n" in a file called `rootfile.txt` by using command `nano rootfile.txt`.
4. I created a directory in "rootdir" called `subdir` by using command `mkdir subdir` and changed into `subdir` by using command `cd subdir`. Then I created a file named `subfile.txt`with the same content by using command `nano subfile.txt`.

[2] Save to S3 by updating cloudstorage.py

I uploaded those files prepared in Step 1 to the newly created S3 bucket by changing the provide Python code to the following.

Python script: `cloudstorage.py`

```
import os
import boto3
import base64

# Define the local root directory to start the file walk
ROOT_DIR = '.'

# Define the root S3 directory (bucket name) to upload files to
ROOT_S3_DIR = '22756463-cloudstorage'

# Initialize the S3 client using Boto3 to interact with AWS S3 service
s3 = boto3.client("s3")

# Define the AWS region for the S3 bucket
bucket_region = 'us-east-1'

# Set the bucket configuration, with the region being the LocationConstraint
bucket_config = {'LocationConstraint': bucket_region}


# Function to upload a file to S3
def upload_file(folder_name, file, file_name):
    print("Uploading %s" % file)  # Print message showing the file being uploaded

    try:
        # Upload the file to the S3 bucket, placing it under the appropriate
folder
        s3.upload_file(file, ROOT_S3_DIR, folder_name+file_name)
```

```
    except Exception as error:
        # Catch and print any errors that occur during the file upload
        print(error)
        pass  # Ignore the error, allowing the script to continue


# Check if the S3 bucket already exists
try:
    # Use head_bucket to check for the existence of the bucket
    s3.head_bucket(Bucket=ROOT_S3_DIR)
    print(f"Bucket {ROOT_S3_DIR} already exists.")  # Inform user if the bucket
already exists
except Exception as error:
    # If the bucket does not exist, create it
    print(f"Creating bucket {ROOT_S3_DIR}...")  # Inform user that the bucket is
being created
    try:
        # Special case: if the region is 'us-east-1', no LocationConstraint is
needed
        if bucket_region == 'us-east-1':
            # Create the bucket without LocationConstraint for the N. Virginia
region
            s3.create_bucket(Bucket=ROOT_S3_DIR)
        else:
            # Create the bucket with LocationConstraint for regions other than
'us-east-1'
            response = s3.create_bucket(
                Bucket=ROOT_S3_DIR,
                CreateBucketConfiguration=bucket_config
            )
            print(f"Bucket {ROOT_S3_DIR} created successfully.")  # Print success
message
    except Exception as error:
        # If an error occurs during bucket creation, print the error message
        print(f"Error creating bucket: {str(error)}")
        raise  # Re-raise the error to terminate the script if bucket creation
fails

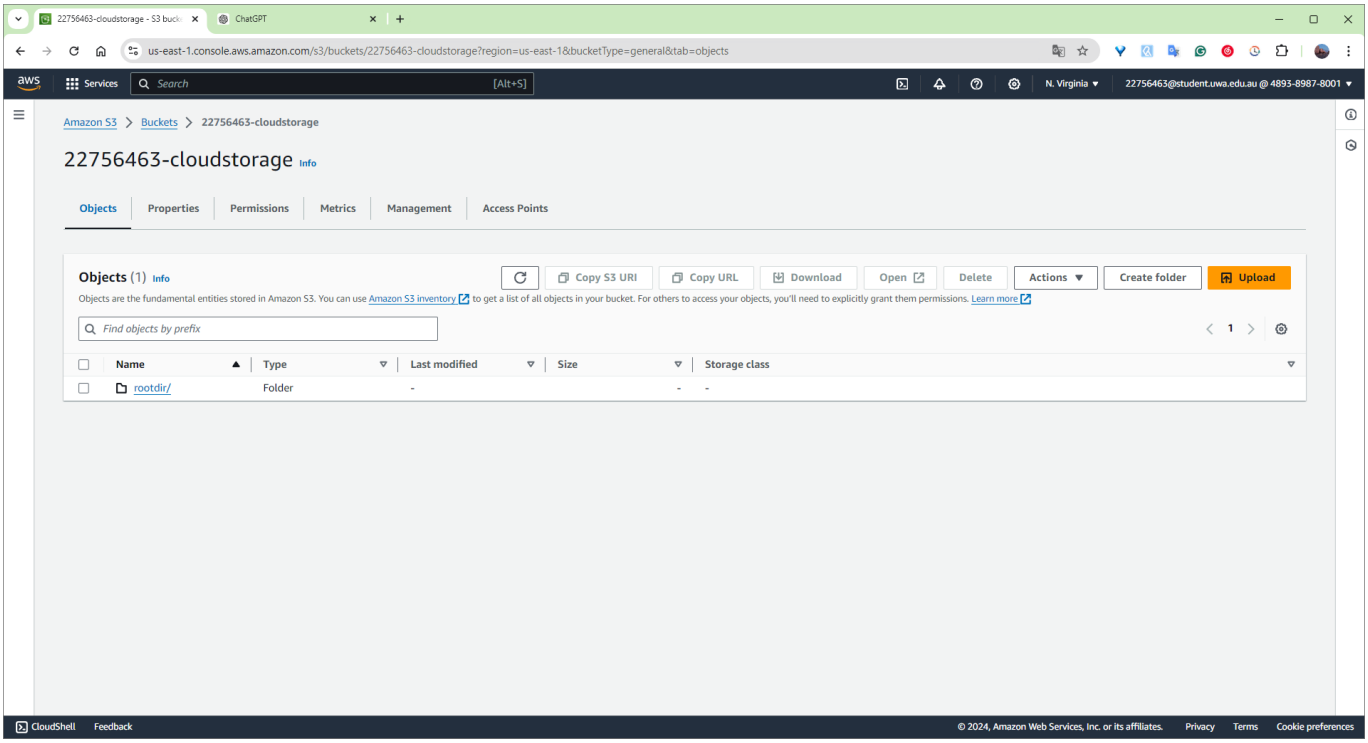

# Parse the local directory and upload files to S3
for dir_name, subdir_list, file_list in os.walk(ROOT_DIR, topdown=True):
    # Ignore the root directory itself and start from its subdirectories
    if dir_name != ROOT_DIR:
        # Loop over each file in the directory
        for fname in file_list:
            # Upload each file to S3 using the folder structure from the local
directory
            upload_file("%s/" % dir_name[2:], "%s/%s" % (dir_name, fname), fname)

# Print done message after all files have been uploaded
print("done")
```

A detailed breakdown is provided as comments above.

Results

Files uploaded to the S3 bucket with the same file structure as shown in Step 1.



[3] Restore from S3

I downloaded the files and directories from the S3 bucket by running the following Python script.

Python script: `restorefromcloud.py`

```
import os  # Import the os module for interacting with the file system
import boto3  # Import the boto3 module for working with AWS services, including
S3

# Initialise the S3 client to interact with AWS S3 service
s3 = boto3.client('s3')

# Define the S3 bucket name from which the files will be restored
BUCKET_NAME = '22756463-cloudstorage'

# Define the root directory where the restored files will be saved locally
LOCAL_ROOT_DIR = './restored_files'

# Loop through each object (file) in the specified S3 bucket
for object in s3.list_objects(Bucket = BUCKET_NAME)['Contents']:
    object_key = object['Key']  # Get the key (path) of the object in the S3
bucket
    print("Downloading %s" % object_key)  # Print the name of the object being
downloaded

    # Get the directory path of the object in the S3 bucket (excluding the file
name)
```

```
    cloud_dir_path = os.path.dirname(object_key)

    # Create the corresponding local directory path to save the file
    local_dir_path = os.path.join(LOCAL_ROOT_DIR, cloud_dir_path)

    # Check if the local directory exists, if not, create it
    if not os.path.exists(local_dir_path):
        os.makedirs(local_dir_path)

    # Define the full local file path to save the downloaded file
    local_file_path = os.path.join(LOCAL_ROOT_DIR, object_key)

    # Download the object from the S3 bucket and save it to the specified local
path
    s3.download_file(BUCKET_NAME, object_key, local_file_path)
```

A detailed breakdown is provided as comments above.

Results

The files and directories from the S3 bucket were downloaded into the local Linux environment.

[4] Write information about files to DynamoDB

1. Install DynamoDB and jre

I created a direcotry named `dynamodb` and changed into that directory by using commands `mkdir dynamodb` and `cd dynamodb`. Then I installed jre (Java Runtime Environment) using command `sudo apt-get install default-jre` and downloaded DynamoDB compressed file using `wget https://s3-ap-northeast-1.amazonaws.com/dynamodb-local-tokyo/dynamodb_local_latest.tar.gz`. I extracted files from the downloaded DynamoDB compressed archive format by using command `tar -zxvf dynamodb_local_latest.tar.gz`.

Note for myself: Amazon DynamoDB is a fully managed, scalable NoSQL database service offering fast, low-latency data access for high-performance applications.

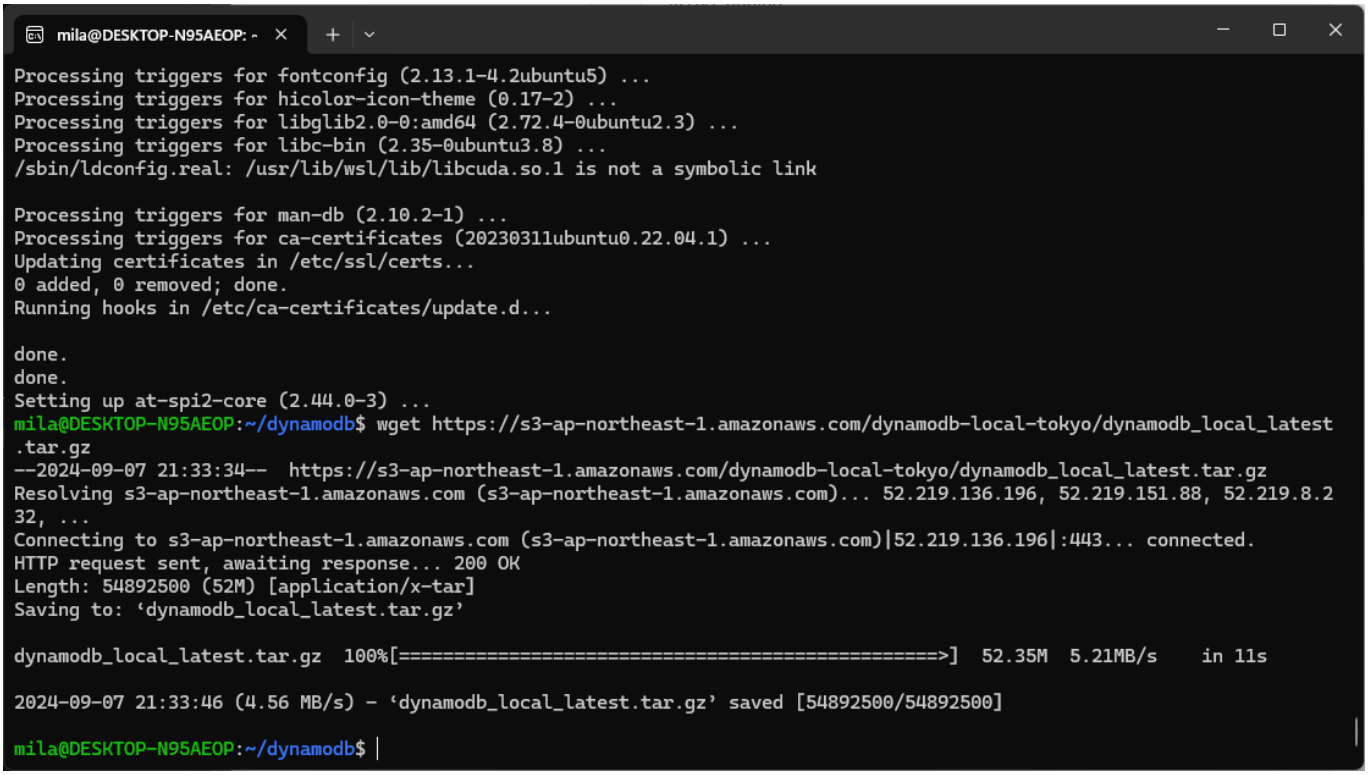
Commands

- `mkdir`: create a directory.
- `cd`: change into another directory.
- `sudo`: Runs the command with superuser (root) privileges.
- `apt-get`: The package manager used to install, update, and manage software packages.
- `install`: The command to install a package.
- `wget`: A command-line tool used to download files from the web. In this case, it's downloading a file named `dynamodb_local_latest.tar.gz` from an Amazon S3 bucket.
- `tar`: Used to create and extract archive files in Linux. In this case, it is used to extract the files from a compressed archive (.tar.gz).
- `-zxvf`: The options following tar specify how the archive should be handled. -z: Tells tar to decompress the file using gzip (since .tar.gz is a gzipped tar archive). -x: Stands for extract. This option tells tar to

extract the contents of the archive. -v: Stands for verbose. This option makes tar list the files being extracted as they are processed, so you can see what is happening. -f: Stands for file. This option specifies the name of the archive file that tar should operate on. In this case, it's dynamodb_local_latest.tar.gz.

Results

DynamoDB and jre were installed in the local Linux environment.



2. Run DynamoDB local

I started DynamoDB Local on my machine with a single shared database for all users and regions, using the native libraries stored in the DynamoDBLocal_lib directory. *It allows me to test DynamoDB operations without connecting to the actual AWS cloud.*

Commands

- `java`: This is the Java runtime command. It is used to execute Java applications (i.e., run .jar files). The DynamoDB Local application is provided as a .jar file.
- `-Djava.library.path=./DynamoDBLocal_lib`: This is a Java option that sets the library path to a specific directory.
  - `-D`: Specifies a system property in Java.
  - `java.library.path`: This property defines where Java looks for native libraries (such as .so files on Linux, .dll files on Windows, or .dylib files on macOS).
  - `=./DynamoDBLocal_lib`: This sets the path to the directory where the native libraries needed for DynamoDB Local are located. The `.` indicates the current directory, and `DynamoDBLocal_lib` is a subdirectory containing the necessary libraries.
- `-jar DynamoDBLocal.jar`: This part tells Java to run a Java Archive (JAR) file. `-jar`: Indicates that the following file is a JAR file to be executed. `DynamoDBLocal.jar`: This is the JAR file containing the DynamoDB Local application, which simulates the DynamoDB service locally.

- `-sharedDb`: This option tells DynamoDB Local to use a single shared database instance instead of creating separate databases for each user account and region.

Results

The DynamoDB local was running.

```
Initializing DynamoDB Local with the following configuration:
Port:   8000
InMemory:       false
Version:        1.25.0
DbPath: null
SharedDb:       false
shouldDelayTransientStatuses:   false
CorsParams:     null
```

3. Write a DynamoDB table

I created a DynamoDB table, extracted attributes of files of the S3 bucket I created in previous steps, and saved those information into the DynamoDB table using the following Python script.

Python script `fileinfo.py`

```
import boto3  # Import the boto3 module for interacting with AWS services
from botocore.exceptions import ClientError  # Import ClientError to handle
exceptions from AWS services
import os  # Import os module to interact with the file system

# Initialise DynamoDB resource and S3 client
dynamodb = boto3.resource('dynamodb', endpoint_url="http://localhost:8000")  #
Local DynamoDB for testing
s3 = boto3.client('s3')  # S3 client to interact with S3 bucket

# Define the S3 bucket name from which files will be read
BUCKET_NAME = '22756463-cloudstorage'

# Define the DynamoDB table name where file information will be stored
TABLE_NAME = 'CloudFiles'

# Function to create a DynamoDB table
def create_cloudfiles_table():
    # Creating a DynamoDB table with partition key 'userId' and sort key
'fileName'
    # Using ProvisionedThroughput to set read/write capacity units
    try:
        table = dynamodb.create_table(
            TableName=TABLE_NAME,  # Table name
            KeySchema=[  # Defines the primary key structure
                {'AttributeName': 'userId', 'KeyType': 'HASH'},  # Partition key
```

```
(userId)
                {'AttributeName': 'fileName', 'KeyType': 'RANGE'},  # Sort key
(fileName)
            ],
            AttributeDefinitions=[  # Attribute definitions for the key schema
                {'AttributeName': 'userId', 'AttributeType': 'S'},  # String type
for userId
                {'AttributeName': 'fileName', 'AttributeType': 'S'},  # String
type for fileName
            ],
            ProvisionedThroughput={'ReadCapacityUnits': 10, 'WriteCapacityUnits':
10}  # Read and write capacity settings
        )
        # Wait until the table is created successfully
        table.meta.client.get_waiter('table_exists').wait(TableName=TABLE_NAME)
        print(f"Table {TABLE_NAME} created successfully.")
        return table  # Return the created table
    except ClientError as e:
        # Handle the case where the table already exists
        if e.response['Error']['Code'] == 'ResourceInUseException':
            print(f"Table {TABLE_NAME} already exists.")
        else:
            raise  # Raise other errors

# Function to get the file attributes for each file in the S3 bucket
def get_s3_file_attributes(bucket_name):
    try:
        # List all objects (files) in the specified S3 bucket
        response = s3.list_objects_v2(Bucket=bucket_name)

        # Check if there are no files in the bucket
        if 'Contents' not in response:
            print(f"No files found in bucket {bucket_name}.")
            return []

        files = []  # List to hold file attribute dictionaries
        # Iterate through each file in the bucket
        for object in response['Contents']:
            s3_key = object['Key']  # Get the key (file path) of the object

            # Get the Access Control List (ACL) for the object to retrieve file
ownership and permission details
            acl_response = s3.get_object_acl(Bucket=bucket_name, Key=s3_key)
            userId = acl_response['Owner']['ID']  # Get the user ID of the owner
            owner = acl_response['Owner']['DisplayName']  # Get the owner's
display name
            permission = acl_response['Grants'][0]['Permission']  # Get the
permission (e.g., FULL_CONTROL)

            # Create a dictionary of file attributes
            file_info = {
                'userId': userId,  # Owner's user ID
                'fileName': s3_key.split('/')[-1],  # File name (last part of the
S3 key)
```

```
                'path': s3_key,  # Full path of the file in the S3 bucket
                'lastUpdated': object['LastModified'].isoformat(),  # Last
modified date (in ISO format)
                'owner': owner,  # Owner's display name
                'permissions': permission  # File permissions
            }
            files.append(file_info)  # Append file info to the list
        return files  # Return the list of file attributes

    except ClientError as e:
        # Handle any errors encountered when accessing the S3 bucket
        print(f"Error accessing S3 bucket: {e}")
        return []

# Function to write file attributes to DynamoDB
def write_to_dynamodb(table_name, files):
    table = dynamodb.Table(table_name)  # Access the DynamoDB table

    # Iterate through each file's attributes and insert them into the table
    for file_info in files:
        try:
            # Insert the file info as a new item in the DynamoDB table
            table.put_item(Item=file_info)
            print(f"Successfully inserted: {file_info['fileName']}")  # Print
success message
        except ClientError as e:
            # Handle errors encountered when inserting data into the table
            print(f"Error inserting {file_info['fileName']} into DynamoDB:
{e.response['Error']['Message']}")

# Main function to orchestrate the process
if __name__ == "__main__":
    # Step 1: Create the DynamoDB table
    create_cloudfiles_table()

    # Step 2: Get the file attributes from the S3 bucket
    files = get_s3_file_attributes(BUCKET_NAME)

    # Step 3: If there are files, write their attributes to the DynamoDB table
    if files:
        write_to_dynamodb(TABLE_NAME, files)
        print("Done.")  # Print a final success message
    else:
        print("No files to insert.")  # Print message if there are no files
```

A detailed explanation is provided as comments above.

Results

Successfully inserted information about `rootfile.txt` and `subfile.txt`.

[5] Scan the table

I scanned and output the table in the command line interface by using command `aws dynamodb scan --table-name CloudFiles --endpoint-url http://localhost:8000`.

Commands

- `aws dynamodb scan`: This scans the entire CloudFiles table and returns all items.
- `--table-name CloudFiles`: Specifies the DynamoDB table I created (CloudFiles).
- `--endpoint-url http://localhost:8000`: This flag is used only if running DynamoDB locally. It specifies the local endpoint.

Results


```
{
    "Items": [
        {
            "owner": {
                "S": "zhi.zhang"
            },
            "path": {
                "S": "rootdir/rootfile.txt"
            },
            "lastUpdated": {
                "S": "2024-09-07T13:00:00+00:00"
            },
            "fileName": {
                "S": "rootfile.txt"
            },
            "userId": {
                "S":
"2a5fac7aada1ad2caa48c9ab08cc4e2428d4eb596108daa3b59f1204ae96482e"
            },
            "permissions": {
                "S": "FULL_CONTROL"
            }
        },
        {
            "owner": {
                "S": "zhi.zhang"
            },
            "path": {
                "S": "rootdir/subdir/subfile.txt"
            },
            "lastUpdated": {
                "S": "2024-09-07T13:00:00+00:00"
            },
            "fileName": {
                "S": "subfile.txt"
            },
            "userId": {
                "S":
"2a5fac7aada1ad2caa48c9ab08cc4e2428d4eb596108daa3b59f1204ae96482e"
            },
```

```
            "permissions": {
                "S": "FULL_CONTROL"
            }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}
```


[6] Delete the table

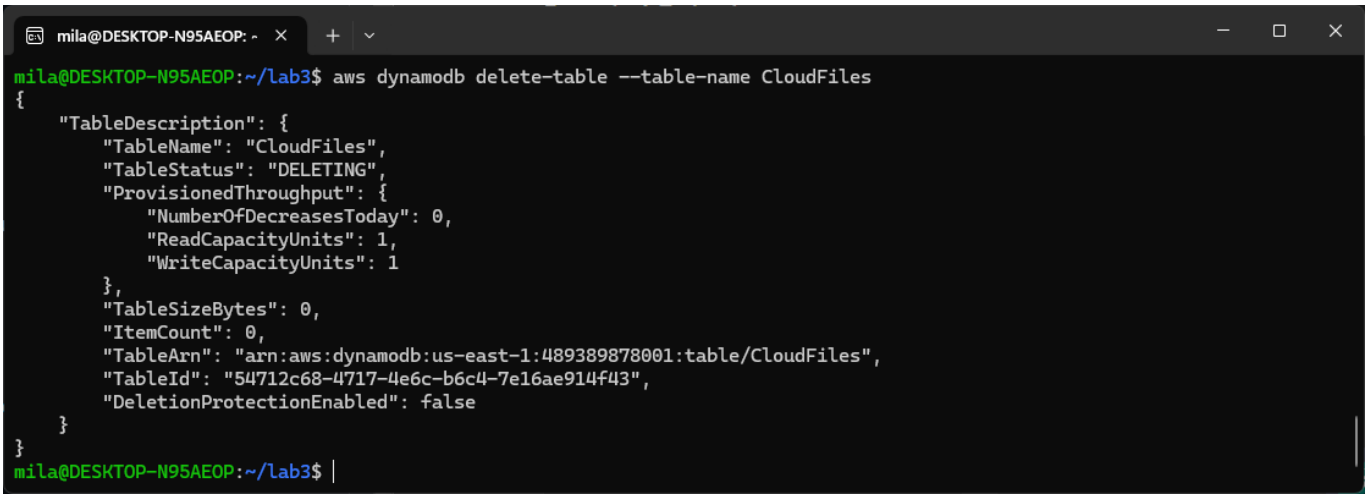
I deleted the created DynamoDB table using command `aws dynamodb delete-table --table-name CloudFiles --endpoint-url http://localhost:8000` , and deleted the created S3 bucket from AWS console.

Commands

- `aws dynamodb delete-table`: This command deletes a DynamoDB table.
- `--table-name CloudFiles`: Specifies the name of the table I want to delete (in this case, CloudFiles).
- `-endpoint-url http://localhost:8000`: If using DynamoDB Local, I need to specify the endpoint URL.

Results

The DynamoDB table is deleted.



Lab 4

Apply a policy to restrict permissions on bucket

I applied a policy on the S3 bucket to restrict access from other users by utilising a Python script as follows.

[1] Write a Python script

Python script `apply_policy.py`

```
import boto3  # Import boto3 module to interact with AWS services
import json  # Import json module to work with JSON data

# Initialise the S3 client to interact with S3 buckets
s3 = boto3.client('s3')

# Specify the S3 bucket name
bucket_name = '22756463-cloudstorage'

# Specify the student number, which will be used in the policy condition
student_number = '22756463'

# Define the bucket policy as a dictionary
bucket_policy = {
    "Version": "2012-10-17",  # Policy version, fixed format
    "Statement": [{  # Policy statement list
        "Sid": "AllowAllS3ActionsInUserFolderForUserOnly",  # Statement ID, just a
unique identifier
        "Effect": "Deny",  # Deny actions unless condition is met
        "Principal": "*",  # Applies to all users
        "Action": "s3:*",  # Deny all S3 actions
        "Resource": f"arn:aws:s3:::{bucket_name}",  # Applies to the specific
bucket
        "Condition": {  # Condition under which the policy applies
            "StringNotLike": {  # Deny if aws:username does not match the
specified pattern
                "aws:username": f"{student_number}@student.uwa.edu.au"  # The
username condition
            }
        }
    }]
}

# Convert the policy dictionary to a JSON string, required by AWS for policies
bucket_policy_json = json.dumps(bucket_policy)

# Function to apply the bucket policy
def apply_bucket_policy(bucket_name, bucket_policy_json):
    try:
        # Use the put_bucket_policy method to apply the policy to the S3 bucket
```

```
        s3.put_bucket_policy(Bucket=bucket_name, Policy=bucket_policy_json)
        print(f"Policy applied successfully to bucket {bucket_name}")  # Success
message
    except Exception as e:
        # Handle any errors encountered when applying the policy
        print(f"Error applying policy: {e}")

# Main execution block
if __name__ == "__main__":
    # Call the function to apply the bucket policy
    apply_bucket_policy(bucket_name, bucket_policy_json)
```

Results

Policy applied to the S3 bucket.

[2] Check whether the script works

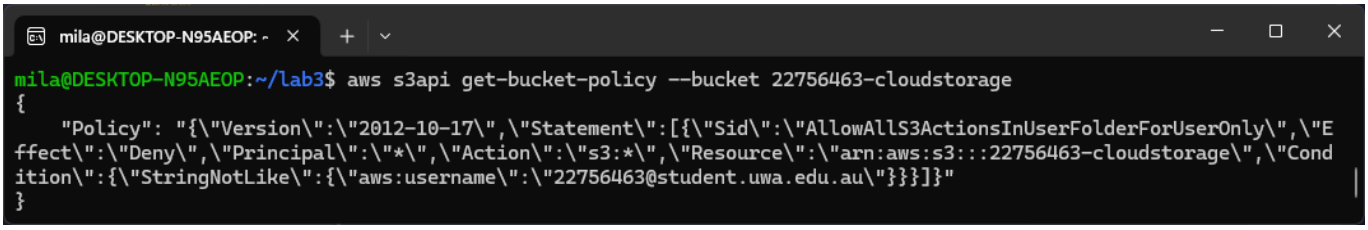
I displayed the policy content applied to the S3 bucket by using AWS CLI command (`aws s3api get-bucket-policy --bucket 22756463-cloudstorage`)and AWS S3 console. Then I checked the effect of the script by asking someone else to access the folder called `rootdir` in the S3 bucket I created.

Commands

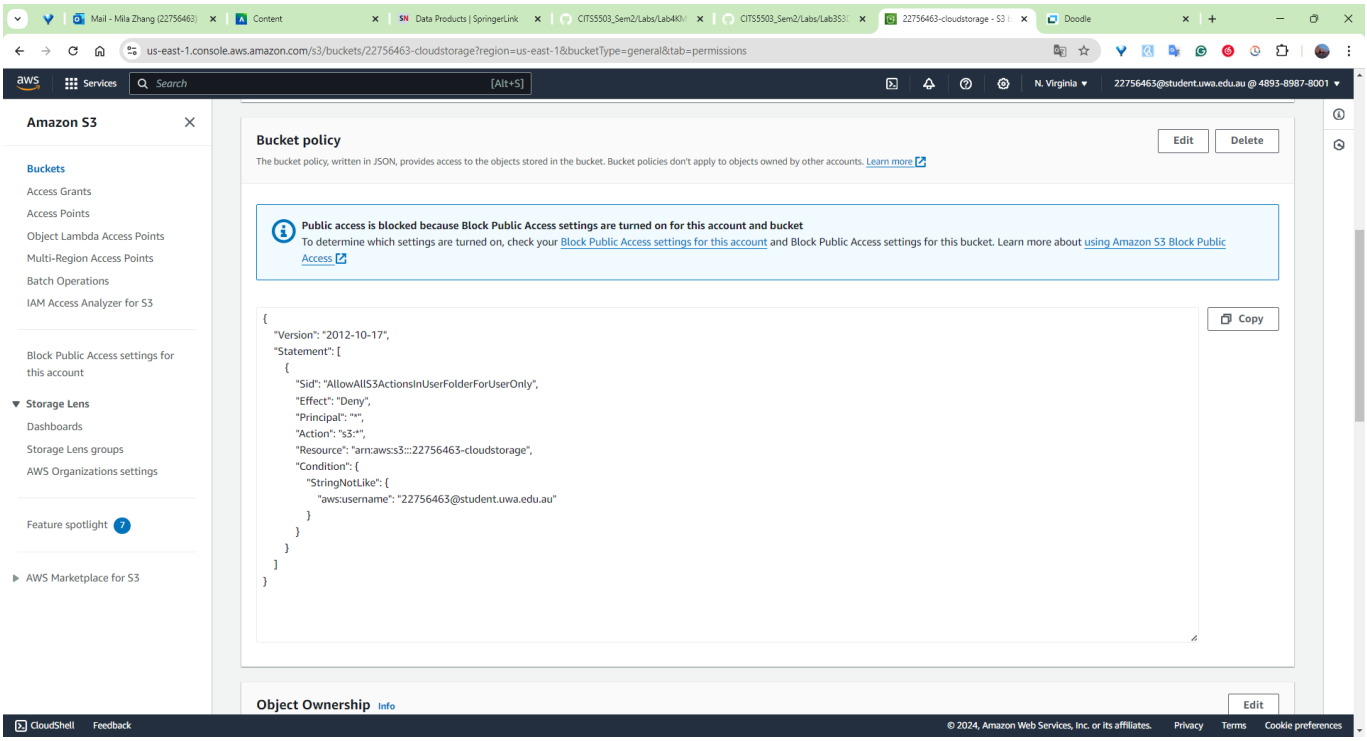
- `aws s3api get-bucket-policy`: This command fetches the policy attached to the specified S3 bucket.
- `--bucket 22756463-cloudstorage`: Specifies the name of the S3 bucket for which you want to retrieve the policy.

Results

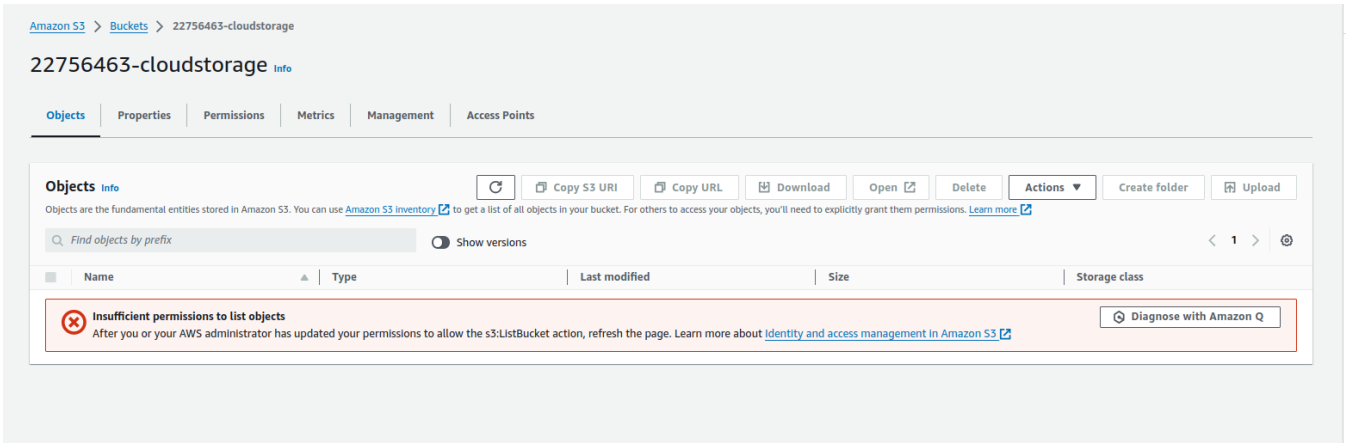
Output from the AWS CLI using my account:



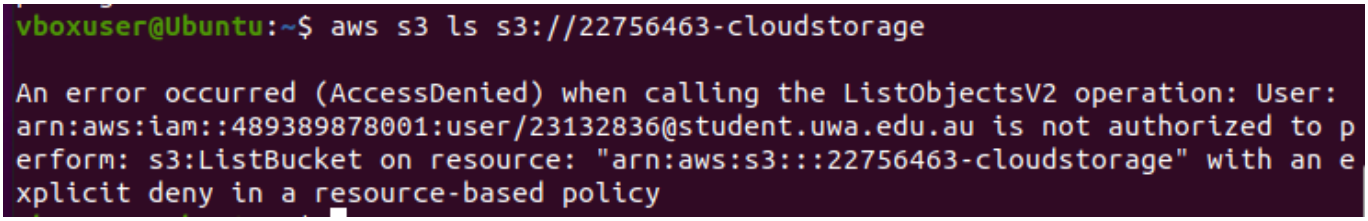
Output from the AWS S3 console using my account:



Output from the AWS S3 console using someone else's account:



Output form the AWS CLI using someone else's account:



AES Encryption using KMS

[1] Create a KMS key

I created a KMS key by running the following Python script.

Python script `create_kms.py`

```
import boto3  # Import boto3 module to interact with AWS services

# Initialise the KMS (Key Management Service) client
kms = boto3.client('kms')

# Define the student number, which will be used in the key alias
student_number = '22756463'

# Create a KMS key
try:
    # Create a new KMS key for encryption and decryption purposes
    response = kms.create_key(
        Description="KMS key for student number",  # Description of the KMS key
        KeyUsage='ENCRYPT_DECRYPT',  # Specify that the key will be used for both
encryption and decryption
    )
    # Extract the KeyId from the response after the key is created
    key_id = response['KeyMetadata']['KeyId']
    print(f"KMS Key created with KeyId: {key_id}")  # Print the KeyId of the
created key
except Exception as e:
    # Handle any errors encountered during key creation
    print(f"Error creating KMS key: {e}")

# Create an alias for the KMS key
try:
    # Define an alias name using the student number
    alias_name = f'alias/{student_number}'
    # Create the alias that points to the newly created KMS key
    kms.create_alias(
        AliasName=alias_name,  # Alias name for the KMS key
        TargetKeyId=key_id  # The KeyId of the previously created KMS key
    )
    print(f"Alias '{alias_name}' created for KMS KeyId: {key_id}")  # Print
confirmation of alias creation
except Exception as e:
    # Handle any errors encountered during alias creation
    print(f"Error creating alias: {e}")
```

A detailed breakdown is provided as comments above.

Results

KMS Key created with KeyId: `b7717fc6-fe96-48a7-bc91-fb6a9d8656a3`. Alias 'alias/22756463' created for KMS KeyId: b7717fc6-fe96-48a7-bc91-fb6a9d8656a3.

[2] Attach a policy to the created KMS key

I updated the Python script in Step 1 and attached a policy to the KMS key created in step 1.

Python script `update_policy.py`

```
import boto3  # Import boto3 module to interact with AWS services
import json  # Import json module to handle JSON data

# Initialise the KMS (Key Management Service) client
kms = boto3.client("kms")

# Replace with your student number to create a unique alias
student_number = "22756463"
alias_name = f"alias/{student_number}"  # Alias for the KMS key

# Create a KMS key
try:
    # Create a new KMS key for encryption and decryption
    response = kms.create_key(
        Description="KMS key for student number",  # A brief description of the
key's purpose
        KeyUsage="ENCRYPT_DECRYPT",  # Specifies that the key will be used for
both encryption and decryption
    )
    # Extract the KeyId of the created key from the response
    key_id = response["KeyMetadata"]["KeyId"]
    print(f"KMS Key created with KeyId: {key_id}")  # Output the KeyId of the
newly created key
except Exception as e:
    # Handle errors that occur during key creation
    print(f"Error creating KMS key: {e}")

# Check if an alias already exists and delete it if it does
try:
    # List all existing aliases in KMS
    response = kms.list_aliases()
    for alias in response['Aliases']:
        # Check if the alias with the same name already exists
        if alias['AliasName'] == alias_name:
            print(f"Alias {alias_name} already exists. Deleting it.")  # Notify
about existing alias
            # Delete the existing alias
            kms.delete_alias(AliasName=alias_name)
            break  # Break out of the loop once the alias is found and deleted
except Exception as e:
    # Handle errors that occur while checking or deleting aliases
    print(f"Error checking or deleting existing alias: {e}")

# Create an alias for the newly created KMS key
try:
    # Create a new alias for the KMS key
    kms.create_alias(AliasName=alias_name, TargetKeyId=key_id)
    print(f"Alias '{alias_name}' created for KMS KeyId: {key_id}")  # Output the
alias creation success message
except Exception as e:
    # Handle errors that occur during alias creation
    print(f"Error creating alias: {e}")
```

```
# Attach a policy to the KMS key
try:
    # Define the key policy as a dictionary
    key_policy = {
        "Version": "2012-10-17",  # Policy version format
        "Id": "key-consolepolicy-3",  # Policy identifier
        "Statement": [
            {
                "Sid": "Enable IAM User Permissions",  # Statement identifier
                "Effect": "Allow",  # Allow permissions
                "Principal": {"AWS": "arn:aws:iam::489389878001:root"},  # The
root user of the AWS account
                "Action": "kms:*",  # Allow all KMS actions
                "Resource": "*",  # Applies to all KMS resources
            },
            {
                "Sid": "Allow access for Key Administrators",  # Statement for key
administrators
                "Effect": "Allow",
                "Principal": {"AWS":
"arn:aws:iam::489389878001:user/22756463@student.uwa.edu.au"},  # Specify the
student user
                "Action": [
                    "kms:Create*", "kms:Describe*", "kms:Enable*", "kms:List*",
"kms:Put*", "kms:Update*",
                    "kms:Revoke*", "kms:Disable*", "kms:Get*", "kms:Delete*",
"kms:TagResource",
                    "kms:UntagResource", "kms:ScheduleKeyDeletion",
"kms:CancelKeyDeletion",
                ],  # Define actions allowed for key administrators
                "Resource": "*",  # Apply to all resources
            },
            {
                "Sid": "Allow use of the key",  # Statement for using the key for
encryption and decryption
                "Effect": "Allow",
                "Principal": {"AWS":
"arn:aws:iam::489389878001:user/22756463@student.uwa.edu.au"},  # Same student
user
                "Action": [
                    "kms:Encrypt", "kms:Decrypt", "kms:ReEncrypt*",
"kms:GenerateDataKey*", "kms:DescribeKey",
                ],  # Define actions allowed for using the key
                "Resource": "*",  # Apply to all resources
            },
            {
                "Sid": "Allow attachment of persistent resources",  # Statement
for attaching resources to the key
                "Effect": "Allow",
                "Principal": {"AWS":
"arn:aws:iam::489389878001:user/22756463@student.uwa.edu.au"},  # Same student
user
                "Action": ["kms:CreateGrant", "kms:ListGrants",
```

```
"kms:RevokeGrant"],  # Actions related to grants
                "Resource": "*",  # Apply to all resources
                "Condition": {"Bool": {"kms:GrantIsForAWSResource": "true"}},  #
Condition for granting AWS resources
            },
        ],
    }

    # Attach the policy to the newly created KMS key
    kms.put_key_policy(
        KeyId=key_id,  # Attach policy to the specified KMS key
        PolicyName="default",  # Use the default policy name
        Policy=json.dumps(key_policy),  # Convert the key policy dictionary to
JSON format
    )
    print(f"Key policy attached to KMS KeyId: {key_id}")  # Output the success
message
except Exception as e:
    # Handle errors that occur when attaching the policy
    print(f"Error attaching key policy: {e}")
```

A detailed explanation is provided as comments above.

Results

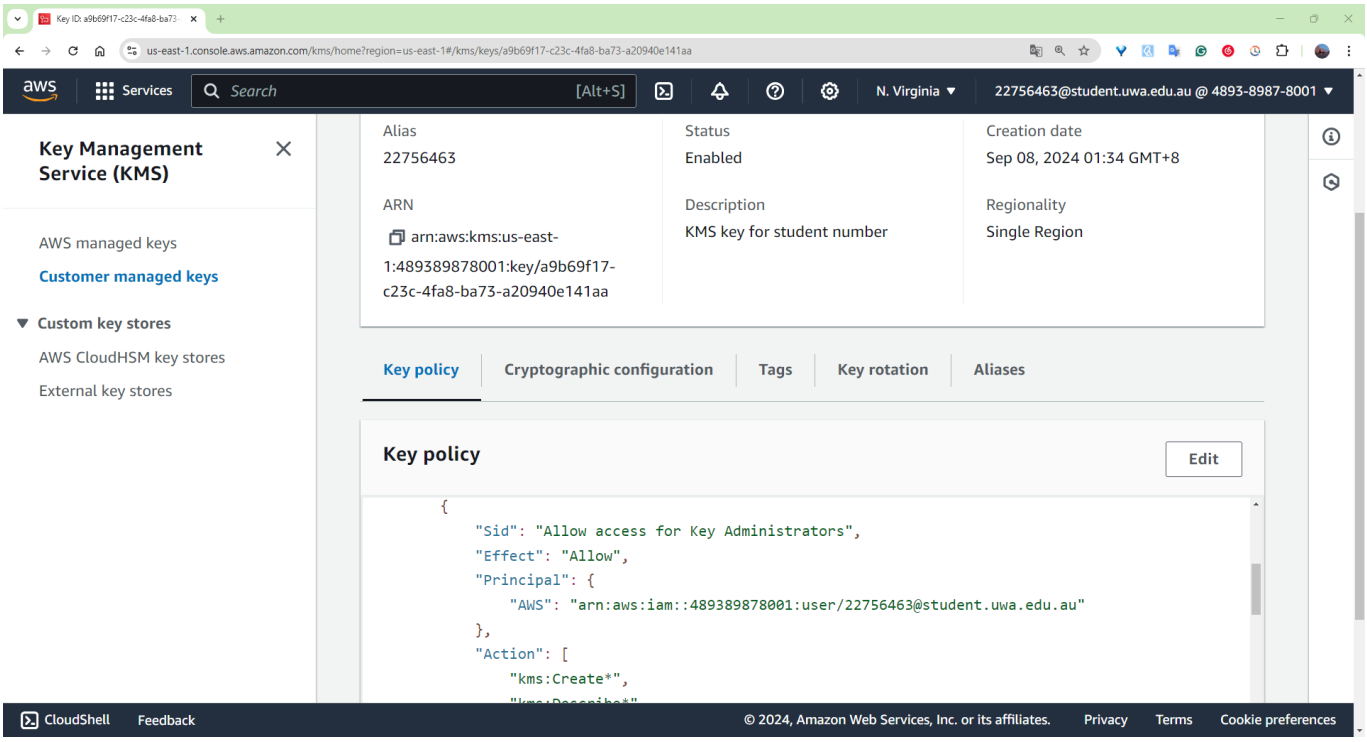
The key policy is attached to the KMS key.

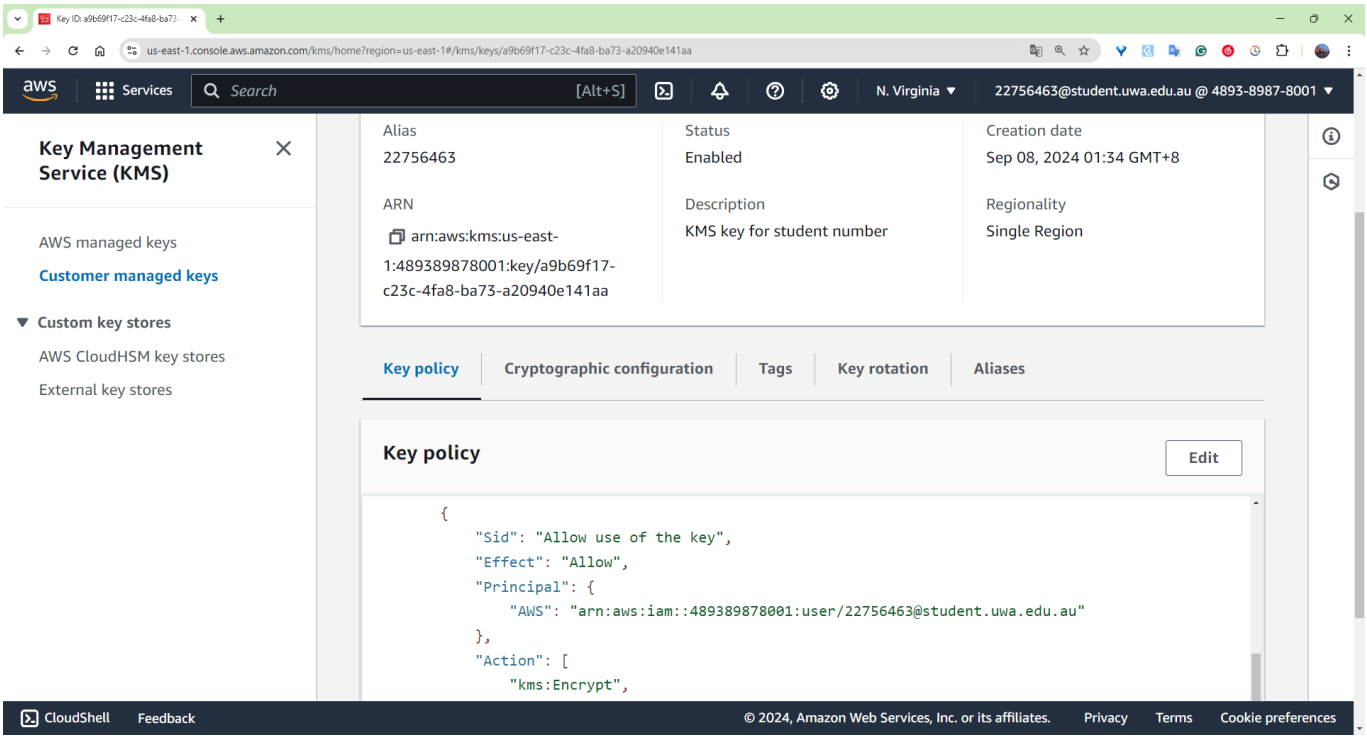
[3] Check whether the script works

1. I logged in to the AWS console and navigated to the KMS dashboard.
2. I clicked on the Customer managed keys on the left panel.
3. I searched for my Key Alias ("alias/22756463") and clicked on the Key ID to view more details.
4. I clicked on the Key Policy tab to view the key policy. I looked for the section in the policy that has the "Allow access for Key Administrators" and "Allow use of the key".

Results

The script worked. My username is the key administrator and key user.





[4] Use the created KMS key for encryption/decryption

I used the following Python script to download files from the S3 bucket, then encrypt and decrpt them. I saved the encrypted and decrypted files in the same folder as the original files.

Python script `encrypt.py`

```
import boto3  # Import boto3 module to interact with AWS services
import os  # Import os module to interact with the file system

# Initialise the clients for KMS (Key Management Service) and S3
```

```
kms = boto3.client('kms')
s3 = boto3.client('s3')

# Define the S3 bucket name and KMS Key ID
bucket_name = '22756463-cloudstorage'  # Name of the S3 bucket
key_id = 'a9b69f17-c23c-4fa8-ba73-a20940e141aa'  # KMS Key ID used for encryption
local_directory = './restored_files'  # Local directory to store files downloaded
from S3

# Step 1: Download files from the S3 bucket to a local directory
def download_s3_files(bucket_name, local_directory):
    # Check if the local directory exists, if not, create it
    if not os.path.exists(local_directory):
        os.makedirs(local_directory)

    # List all files in the specified S3 bucket
    objects = s3.list_objects_v2(Bucket=bucket_name)

    # Check if the bucket contains any files
    if 'Contents' in objects:
        # Loop through each object (file) in the S3 bucket
        for obj in objects['Contents']:
            file_key = obj['Key']  # Get the S3 file key (path)
            file_path = os.path.join(local_directory, file_key)  # Create the
local file path

            # Create necessary subdirectories for the local path
            os.makedirs(os.path.dirname(file_path), exist_ok=True)

            # Download the file from S3 to the local directory
            s3.download_file(bucket_name, file_key, file_path)
            print(f"Downloaded {file_key} to {file_path}")  # Output download
success message
    else:
        # Print message if no files are found in the bucket
        print("No files found in the S3 bucket.")

# Step 2: Encrypt the file using KMS
def encrypt_file(file_path, key_id):
    # Open the file in binary mode and read its contents
    with open(file_path, 'rb') as file:
        file_data = file.read()

    # Encrypt the file data using KMS
    encrypted_data = kms.encrypt(KeyId=key_id, Plaintext=file_data)
['CiphertextBlob']

    # Write the encrypted data to a new file with ".encrypted" extension
    encrypted_file_path = f"{file_path}.encrypted"
    with open(encrypted_file_path, 'wb') as encrypted_file:
        encrypted_file.write(encrypted_data)

    # Output success message for encryption
    print(f"Encrypted file written to {encrypted_file_path}")
```

```
    return encrypted_file_path  # Return the path of the encrypted file

# Step 3: Decrypt the encrypted file using KMS
def decrypt_file(encrypted_file_path):
    # Open the encrypted file in binary mode and read its contents
    with open(encrypted_file_path, 'rb') as encrypted_file:
        encrypted_data = encrypted_file.read()

    # Decrypt the encrypted data using KMS
    decrypted_data = kms.decrypt(CiphertextBlob=encrypted_data)['Plaintext']

    # Write the decrypted data to a new file with ".decrypted" extension
    decrypted_file_path = f"{encrypted_file_path}.decrypted"
    with open(decrypted_file_path, 'wb') as decrypted_file:
        decrypted_file.write(decrypted_data)

    # Output success message for decryption
    print(f"Decrypted file written to {decrypted_file_path}")
    return decrypted_file_path  # Return the path of the decrypted file

# Main function to orchestrate downloading, encrypting, and decrypting files
def main():
    # Step 1: Download the files from the S3 bucket
    download_s3_files(bucket_name, local_directory)

    # Step 2: Encrypt and decrypt each file in the local directory
    # Walk through the directory and its subdirectories
    for dir_name, subdir_list, file_list in os.walk(local_directory,
topdown=True):
        if dir_name != local_directory:  # Skip the root directory
            for fname in file_list:  # Loop through all files in the current
directory
                file_path = os.path.join(dir_name, fname)  # Construct full file
path

                # Encrypt the file
                encrypted_file_path = encrypt_file(file_path, key_id)

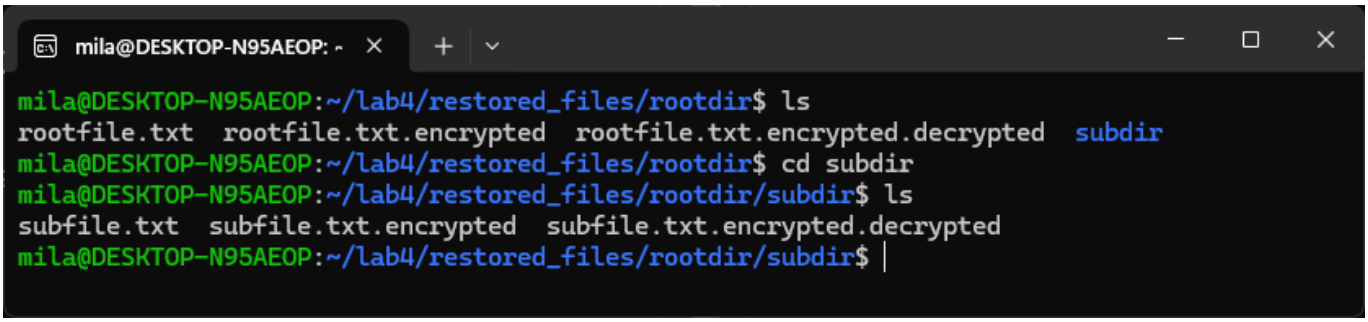
                # Decrypt the encrypted file
                decrypt_file(encrypted_file_path)

# Entry point of the script
if __name__ == "__main__":
    main()  # Call the main function
```

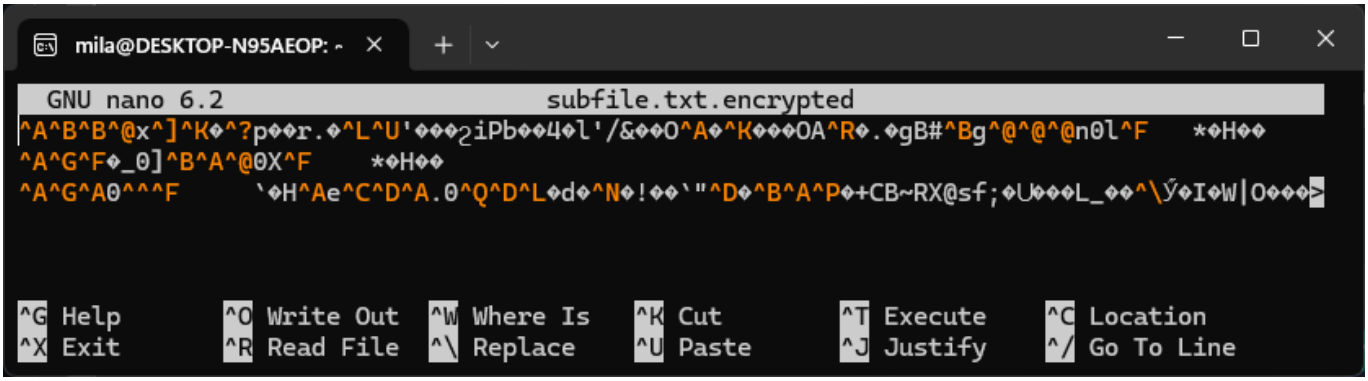
A detailed breakdown is provided above.

Results

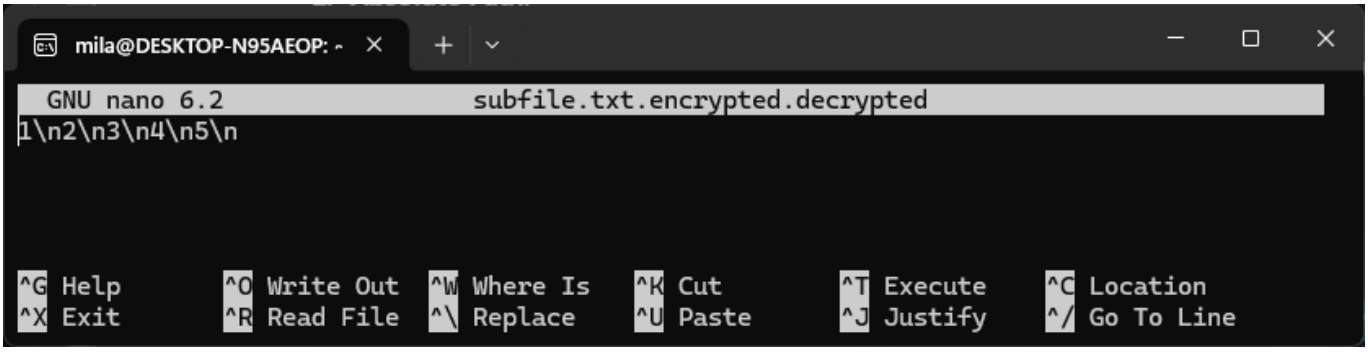
Files are downloaded from the S3 bucket, encrypted and decrypted using the KMS key I created (and updated the key policy) in Step 1 and 2.



The encrypted file:



The decrypted file:



[5] Apply pycryptodome for encryption/decryption

5.1 Install `pycryptodome`

I first installed the library `pycryptodome` by using command `pip install pycryptodome`.

Commands

- `pip`: Pip is the Python package installer. It's used to install, update, or uninstall Python packages.
- `install`: This command tells pip to install a specific package.
- `pycryptodome`: The name of the Python package that provides cryptographic functionalities (e.g., encryption, decryption, hashing).

Results

```
Defaulting to user installation because normal site-packages is not writeable
Collecting pycryptodome
  Downloading pycryptodome-3.20.0-cp35-abi3-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.1 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 6.1 MB/s eta 0:00:00
```

```
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.20.0
```

5.2 Encrypt and decrypt using Python script

I adapted the Python script provided in the Lab4 instruction. I downloaded files from the S3 bucket, encrypted and decrypted them using `pycryptodome`.

Python script `pycryptodome.py`

```
# Reference:
https://github.com/zhangzhics/CITS5503_Sem2/blob/master/Labs/src/fileencrypt.py
import os, random, struct  # Import necessary modules for file handling and random
data generation
from Crypto.Cipher import AES  # Import AES cipher for encryption and decryption
from Crypto import Random  # Import Random module for generating random
initialization vector (IV)
import boto3  # Import boto3 to interact with AWS services like S3
import base64  # Import base64 for encoding/decoding (not used here but commonly
used in encryption)
import hashlib  # Import hashlib to generate a secure hash of the password

# Constants
BLOCK_SIZE = 16  # AES block size (in bytes)
CHUNK_SIZE = 64 * 1024  # Size of chunks (64 KB) for reading and
encrypting/decrypting files

# Initialise the S3 client
s3 = boto3.client('s3')

# Define the S3 bucket name and local directory for storing downloaded files
bucket_name = '22756463-cloudstorage'  # S3 bucket name
local_directory = './restored_files1'  # Local directory to store files obtained
from cloud

# Function to download files from S3 bucket to local directory
def download_s3_files(bucket_name, local_directory):
    # Check if the local directory exists, if not, create it
    if not os.path.exists(local_directory):
        os.makedirs(local_directory)

    # List files in the S3 bucket
    objects = s3.list_objects_v2(Bucket=bucket_name)

    # Check if the bucket contains any files
    if 'Contents' in objects:
        # Loop through each object (file) in the S3 bucket
        for obj in objects['Contents']:
            file_key = obj['Key']  # Get the file key (path) from S3
            file_path = os.path.join(local_directory, file_key)  # Create the
local file path
```

```
            # Create necessary subdirectories in the local path
            os.makedirs(os.path.dirname(file_path), exist_ok=True)

            # Download the file from S3 to local directory
            s3.download_file(bucket_name, file_key, file_path)
            print(f"Downloaded {file_key} to {file_path}")  # Output success
message for each download
    else:
        # Print message if no files are found in the S3 bucket
        print("No files found in the S3 bucket.")

# Function to encrypt a file using AES encryption
def encrypt_file(password, in_filename, out_filename):
    # Generate a 256-bit key from the password using SHA-256 hashing
    key = hashlib.sha256(password.encode("utf-8")).digest()

    # Generate a random initialization vector (IV)
    iv = Random.new().read(AES.block_size)
    encryptor = AES.new(key, AES.MODE_CBC, iv)  # Create AES encryptor in CBC mode
    filesize = os.path.getsize(in_filename)  # Get the original file size

    # Open the input file for reading in binary mode and the output file for
writing
    with open(in_filename, 'rb') as infile:
        with open(out_filename, 'wb') as outfile:
            outfile.write(struct.pack('<Q', filesize))  # Write the file size to
the encrypted file
            outfile.write(iv)  # Write the initialization vector (IV) to the
encrypted file

            # Read and encrypt the file data in chunks
            while True:
                chunk = infile.read(CHUNK_SIZE)
                if len(chunk) == 0:  # If the chunk is empty, stop reading
                    break
                elif len(chunk) % 16 != 0:  # If the chunk is not a multiple of
16, pad it
                    chunk += ' '.encode("utf-8") * (16 - len(chunk) % 16)

                # Write the encrypted chunk to the output file
                outfile.write(encryptor.encrypt(chunk))

# Function to decrypt an encrypted file using AES encryption
def decrypt_file(password, in_filename, out_filename):
    # Generate a 256-bit key from the password using SHA-256 hashing
    key = hashlib.sha256(password.encode("utf-8")).digest()

    # Open the encrypted input file and the output file for the decrypted content
    with open(in_filename, 'rb') as infile:
        origsize = struct.unpack('<Q', infile.read(struct.calcsize('Q')))[0]  #
Read the original file size
        iv = infile.read(16)  # Read the initialization vector (IV) from the
encrypted file
```

```
        decryptor = AES.new(key, AES.MODE_CBC, iv)  # Create AES decryptor in CBC
mode

        # Open the output file to write decrypted data
        with open(out_filename, 'wb') as outfile:
            # Read and decrypt the file data in chunks
            while True:
                chunk = infile.read(CHUNK_SIZE)
                if len(chunk) == 0:  # If the chunk is empty, stop reading
                    break
                # Write the decrypted chunk to the output file
                outfile.write(decryptor.decrypt(chunk))

            # Truncate the file to the original size (removes any padding)
            outfile.truncate(origsize)

# Main program
password = 'cat is cuteee'  # Define a password for encryption and decryption

# Step 1: Download the files from the S3 bucket
download_s3_files(bucket_name, local_directory)

# Step 2: Encrypt and decrypt each file in the local directory
# Walk through the local directory and its subdirectories
for dir_name, subdir_list, file_list in os.walk(local_directory, topdown=True):
    if dir_name != local_directory:  # Skip the root directory
        for fname in file_list:  # Loop through all files in the current directory
            print(fname)  # Print the file name
            file_path = os.path.join(dir_name, fname)  # Construct full file path

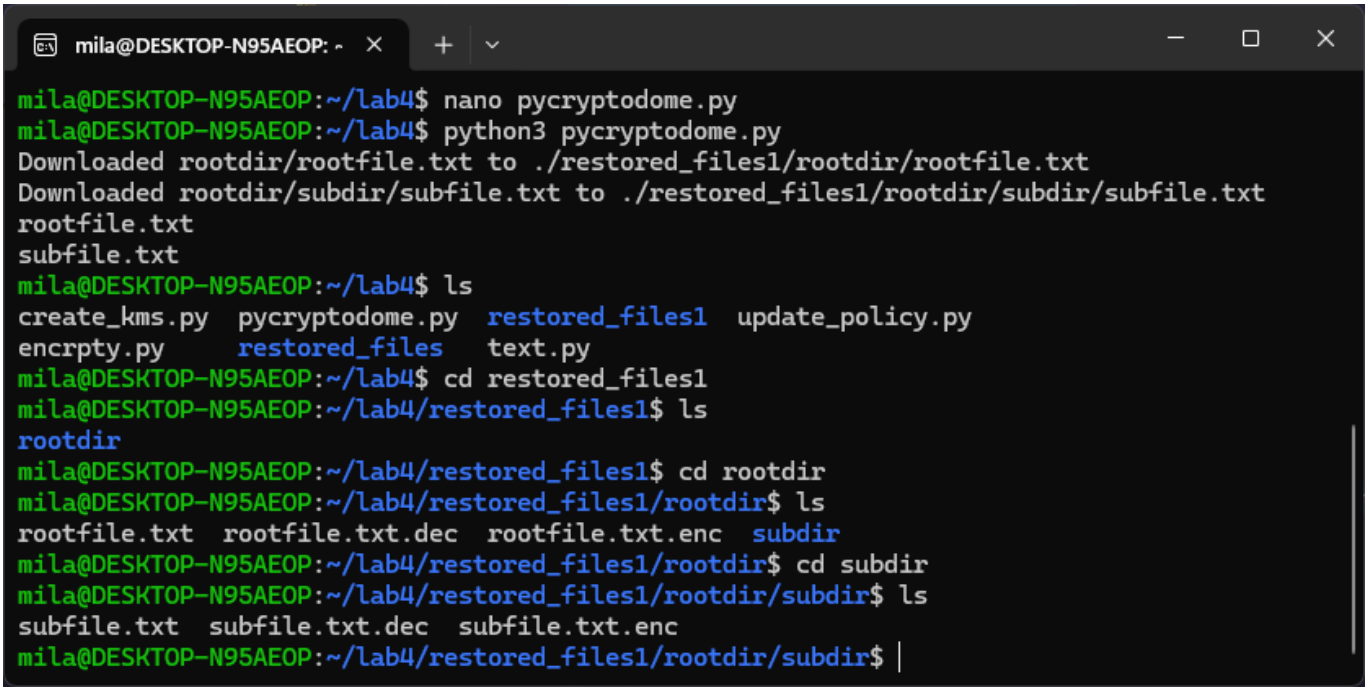
            # Encrypt the file and save the encrypted file with ".enc" extension
            encrypt_file(password, file_path, out_filename=f"{file_path}.enc")

            # Decrypt the encrypted file and save the decrypted file with ".dec"
extension
            decrypt_file(password, f"{file_path}.enc", out_filename=f"
{file_path}.dec")
```

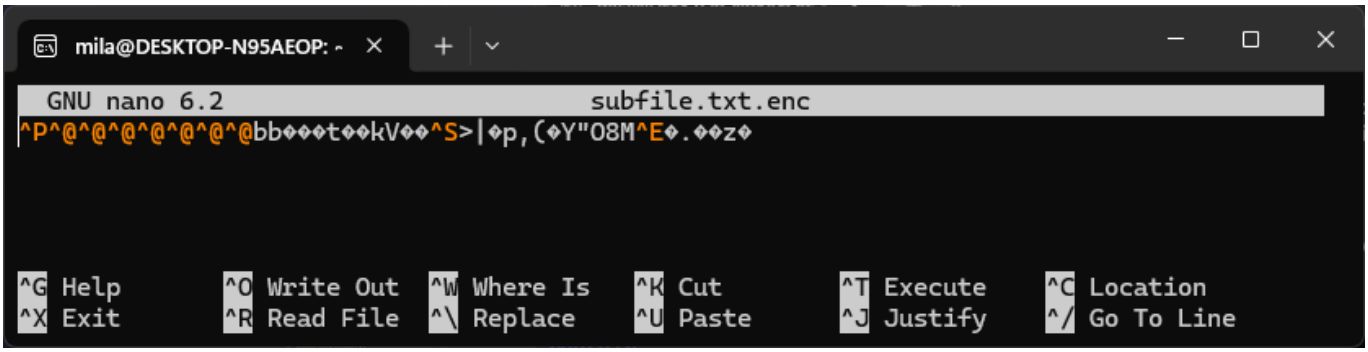
A detailed explanation is provided as comments above.

Results

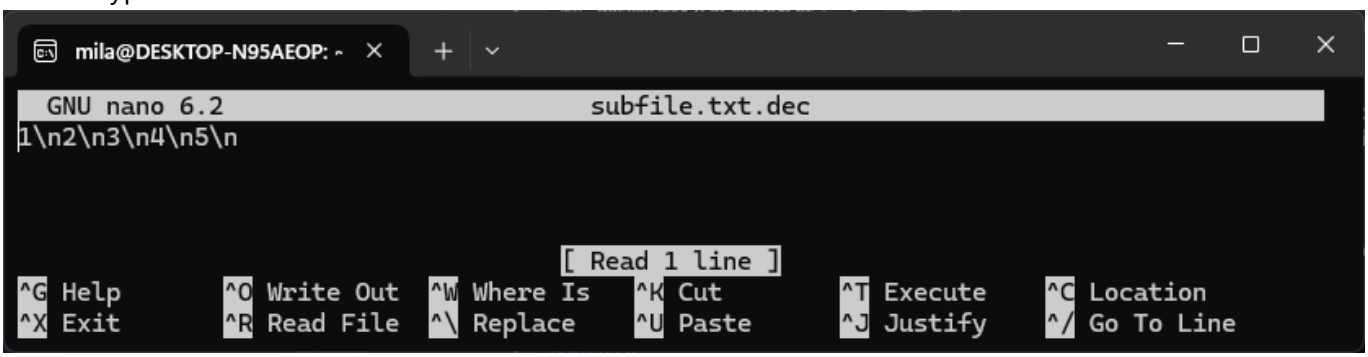
Files were downloaded from the S3 bucket, encrypted and decrypted using the `pycryptodome`.



The encrypted file:



The decrypted file:



Answer the following question

Question:

What is the performance difference between using KMS and using the custom solution?

Answer:

- Latency: Using KMS involves network communication with the AWS service and thus the latency can be higher than the custom solution which operates locally and requires no network.
- Ease of use: Using KMS is easier because AWS handles the key storage and mangement, whereas in the custom solution we need to manage the key (password) ourselves, and the code implementation is

more complicated.

- Security: Using KMS is more secure because the AWS modules handle encryption automatically, whereas in the custom solution requires us to define the password ourselves and if it is chosen unwisely, the encryption is less secure.
- Cost: Using KMS can be more costly because it is charged based on the number of encryption and decryption requests, whereas the custom solution requires no recurring costs each encryption or decryption operation.

Lab 5

Application Load Balancer

[1] Create 2 EC2 instances

I created 2 EC2 instances in two different availability zones in my allocated region(us-east-1) using the Python script below. I also created a security group to authorise inbound traffic for HTTP and SSH before creating the instances.

Python script `create_instances.py`

```
import boto3
from botocore.exceptions import ClientError

# Create a Boto3 EC2 client
ec2 = boto3.client('ec2')

# Step 1: Create a security group
def create_security_group(group_name, description):
    try:
        #
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2/cli
ent/create_security_group.html
        response = ec2.create_security_group(
            GroupName=group_name,
            Description=description,
        )
        security_group_id = response['GroupId']
        print(f'Security Group {group_name} created with ID: {security_group_id}')
        return security_group_id
    except ClientError as e:
        print(f"Error creating security group: {e}")
        return None

# Step 2: Authorise inbound traffic for SSH and HTTP
def authorize_security_group_ingress(group_name):
    try:
        #
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2/cli
ent/authorize_security_group_ingress.html
        ec2.authorize_security_group_ingress(
            GroupName= group_name,
            IpPermissions=[
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 22, # SSH
                    'ToPort': 22,
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
                },
```

```
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 80,  # HTTP
                    'ToPort': 80,
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
                }
            ]
        )
        print(f"SSH and HTTP rule added to Security Group: {group_name}")
    except ClientError as e:
        print(f"Error adding SSH rule to security group: {e}")

# Step 3: Get the available zones in your region
def get_availability_zones():
    availability_zones = ec2.describe_availability_zones()['AvailabilityZones']

    # Make sure there are at least 2 zones available
    if len(availability_zones) < 2:
        raise Exception('Less than 2 availability zones available in this
region.')

    return availability_zones

# Step 4: Create a key pair
def create_key_pair(key_name):
    try:
        #
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2/cli
ent/create_key_pair.html
        key_pair = ec2.create_key_pair(KeyName=key_name)
        private_key = key_pair['KeyMaterial']

        # Save the private key to a .pem file
        with open(f"{key_name}.pem", "w") as key_file:
            key_file.write(private_key)

        print(f"Key pair {key_name} created and saved as {key_name}.pem")

    except ClientError as e:
        print(f"Error creating key pair: {e}")

# Step 5: Create two instances in different availability zones
def create_instances(key_name, security_group_name, ami_id, availability_zones,
student_number):
    for i in range(2):
        try:
            #
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2/cli
ent/run_instances.html
            instances = ec2.run_instances(
                ImageId= ami_id,
                MinCount=1,
                MaxCount=1,
                InstanceType='t2.micro',
```

```
                KeyName=key_name,
                SecurityGroups=[security_group_name],
                Placement={
                    'AvailabilityZone': availability_zones[i]['ZoneName']
                },
                TagSpecifications=[
                    {
                        'ResourceType': 'instance',
                        'Tags': [
                            {'Key': 'Name', 'Value': f'{student_number}-vm{i+1}'}
                        ]
                    }
                ]
            )
            instance_id = instances['Instances'][0]['InstanceId']
            print(f"EC2 instance created with Instance ID: {instance_id}")
        except ClientError as e:
            print(f"Error creating EC2 instance: {e}")

def main():
    sg_name = "22756463-sg"
    sg_description = "security group for development environment"
    student_number = '22756463'
    ami_id = 'ami-0a0e5d9c7acc336f1'
    key_name = "22756463-key"

    # Step 1: Create a security group
    create_security_group(sg_name, sg_description)

    # Step 2: Authorise inbound traffic for SSH
    authorize_security_group_ingress(sg_name)

    # Step 3: Get the available zones in your region
    availability_zones = get_availability_zones()

    # Step 4: Create a key pair
    create_key_pair(key_name)

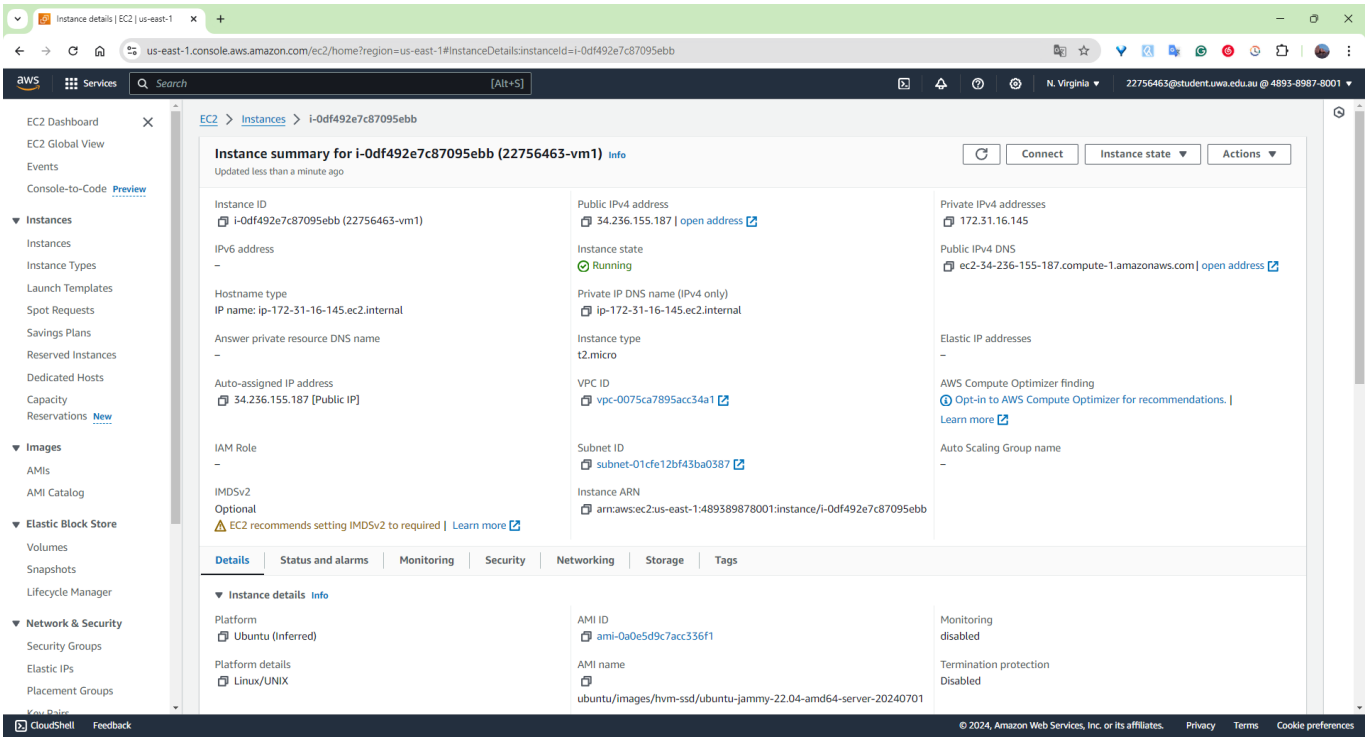
    # Step 5: Create two instances in different availability zones
    create_instances(key_name, sg_name, ami_id, availability_zones,
student_number)

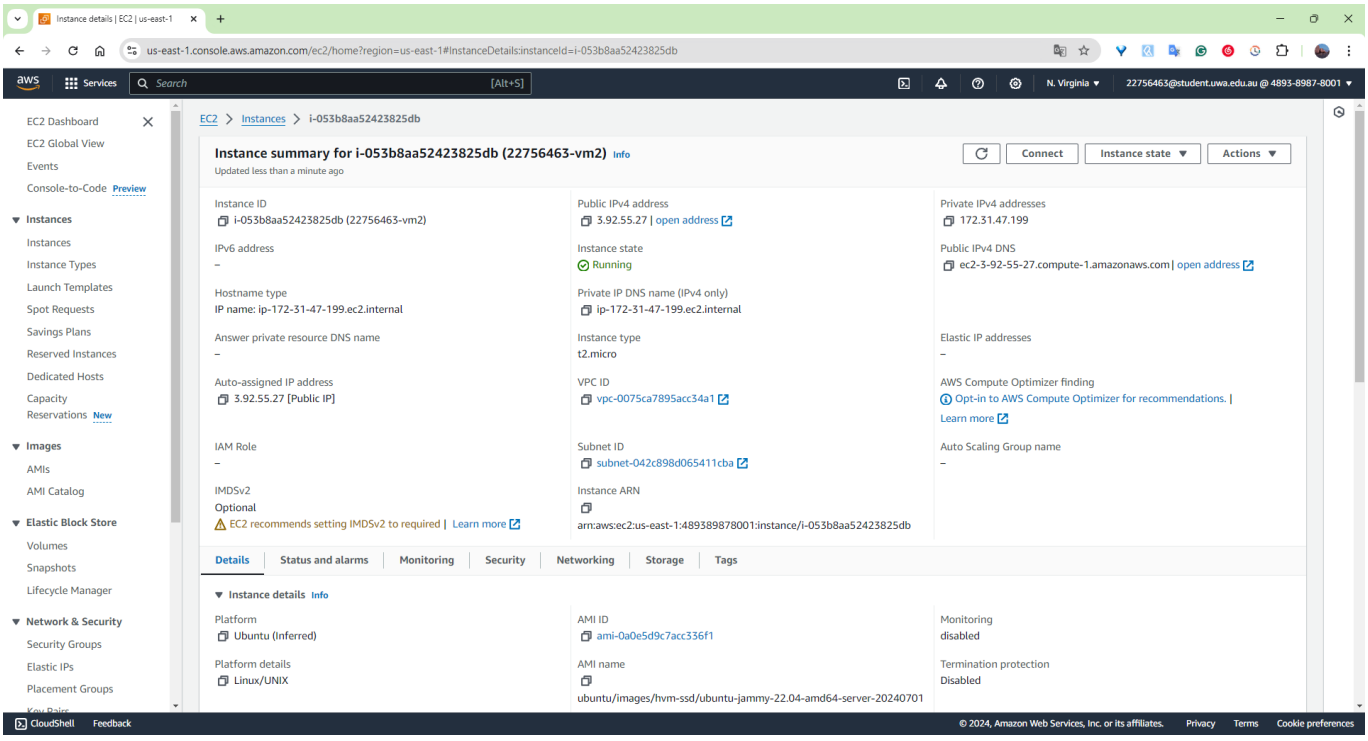
if __name__ == "__main__":
    main()
```

A detailed description of each function can be found in Step 2 Create an Application Load Balancer below.

Results

Two instances created with instance ID: `i-0df492e7c87095ebb` and `i-053b8aa52423825dbas` as follows.





[2] Create an Application Load Balancer

Python script `create_alb.py`

```
import boto3
from botocore.exceptions import ClientError

# Create a Boto3 EC2 and ELBv2 client
ec2 = boto3.client('ec2')
elbv2 = boto3.client('elbv2')
```

```
# Step 1: Create a security group
def create_security_group(group_name, description):
    try:
        response = ec2.create_security_group(
            GroupName=group_name,
            Description=description,
        )
        security_group_id = response['GroupId']
        print(f'Security Group {group_name} created with ID: {security_group_id}')
        return security_group_id
    except ClientError as e:
        print(f"Error creating security group: {e}")
        return None

# Step 2: Authorise inbound traffic for SSH and HTTP
def authorize_security_group_ingress(group_name):
    try:
        ec2.authorize_security_group_ingress(
            GroupName= group_name,
            IpPermissions=[
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 22, # SSH
                    'ToPort': 22,
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
                },
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 80,  # HTTP
                    'ToPort': 80,
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
                }
            ]
        )
        print(f"SSH and HTTP rule added to Security Group: {group_name}")
    except ClientError as e:
        print(f"Error adding rules to security group: {e}")

# Step 3: Get the available zones in your region
def get_availability_zones():
    availability_zones = ec2.describe_availability_zones()['AvailabilityZones']

    if len(availability_zones) < 2:
        raise Exception('Less than 2 availability zones available in this
region.')

    return availability_zones

# Step 4: Create a key pair
def create_key_pair(key_name):
    try:
        key_pair = ec2.create_key_pair(KeyName=key_name)
        private_key = key_pair['KeyMaterial']
        with open(f"{key_name}.pem", "w") as key_file:
```

```
            key_file.write(private_key)
        print(f"Key pair {key_name} created and saved as {key_name}.pem")
    except ClientError as e:
        print(f"Error creating key pair: {e}")

# Step 5: Create two instances in different availability zones
def create_instances(key_name, security_group_name, ami_id, availability_zones,
student_number):
    instance_ids = []
    for i in range(2):
        try:
            instances = ec2.run_instances(
                ImageId= ami_id,
                MinCount=1,
                MaxCount=1,
                InstanceType='t2.micro',
                KeyName=key_name,
                SecurityGroups=[security_group_name],
                Placement={
                    'AvailabilityZone': availability_zones[i]['ZoneName']
                },
                TagSpecifications=[
                    {
                        'ResourceType': 'instance',
                        'Tags': [
                            {'Key': 'Name', 'Value': f'{student_number}-vm{i+1}'}
                        ]
                    }
                ]
            )
            instance_id = instances['Instances'][0]['InstanceId']
            instance_ids.append(instance_id)
            print(f"EC2 instance created with Instance ID: {instance_id}")
        except ClientError as e:
            print(f"Error creating EC2 instance: {e}")
    return instance_ids

# Step 6: Get the available subnets in your region
def get_subnets():
    subnets_response = ec2.describe_subnets()
    subnets = subnets_response['Subnets']

    # Filter subnets to only include those that are in available state
    available_subnets = [subnet for subnet in subnets if subnet['State'] ==
'available']

    if len(available_subnets) < 2:
        raise Exception('Less than 2 available subnets in this region.')

    return available_subnets

# Step 7: Create the network load balancer
def create_load_balancer(sg_id, subnets):
    try:
```

```
        response = elbv2.create_load_balancer(
            Name='22756463-load-balancer',
            Subnets=subnets,
            SecurityGroups=[sg_id],
            Scheme='internet-facing',
            Type='application',
            IpAddressType='ipv4'
        )
        lb_arn = response['LoadBalancers'][0]['LoadBalancerArn']
        print(f"Load Balancer created with ARN: {lb_arn}")
        return lb_arn
    except ClientError as e:
        print(f"Error creating load balancer: {e}")
        return None

# Step 8: Create a target group
def create_target_group(vpc_id):
    try:
        response = elbv2.create_target_group(
            Name='22756463-target-group',
            Protocol='HTTP',
            Port=80,
            VpcId=vpc_id,
            HealthCheckProtocol='HTTP',
            HealthCheckPort='80',
            TargetType='instance'
        )
        target_group_arn = response['TargetGroups'][0]['TargetGroupArn']
        print(f"Target group created with ARN: {target_group_arn}")
        return target_group_arn
    except ClientError as e:
        print(f"Error creating target group: {e}")
        return None

# Step 9: Register targets (EC2 instances) to the target group
def register_targets(target_group_arn, instance_ids):
    try:
        targets = [{'Id': instance_id} for instance_id in instance_ids]
        elbv2.register_targets(
            TargetGroupArn=target_group_arn,
            Targets=targets
        )
        print(f"Registered instances {instance_ids} to the target group.")
    except ClientError as e:
        print(f"Error registering targets: {e}")

# Step 10: Create a listener to forward traffic to the target group
def create_listener(lb_arn, target_group_arn):
    try:
        elbv2.create_listener(
            LoadBalancerArn=lb_arn,
            Protocol='HTTP',
            Port=80,
            DefaultActions=[{
```

```
                'Type': 'forward',
                'TargetGroupArn': target_group_arn
            }]
        )
        print(f"Listener created for Load Balancer {lb_arn}")
    except ClientError as e:
        print(f"Error creating listener: {e}")

def main():
    sg_name = "22756463-sg"
    sg_description = "security group for development environment"
    student_number = '22756463'
    ami_id = 'ami-0a0e5d9c7acc336f1'
    key_name = "22756463-key"

    # Step 1: Create a security group
    sg_id = create_security_group(sg_name, sg_description)

    # Step 2: Authorise inbound traffic for SSH and HTTP
    authorize_security_group_ingress(sg_name)

    # Step 3: Get the available zones in your region
    availability_zones = get_availability_zones()

    # Step 4: Create a key pair
    create_key_pair(key_name)

    # Step 5: Create two instances in different availability zones
    instance_ids = create_instances(key_name, sg_name, ami_id, availability_zones,
student_number)

    # Step 6: Get VPC ID
    vpcs = ec2.describe_vpcs()
    vpc_id = vpcs['Vpcs'][0]['VpcId']

    # Step 7: Create an Application Load Balancer (ALB)
    subnets = get_subnets()
    subnet_ids = [subnets[0]['SubnetId'], subnets[1]['SubnetId']]
    lb_arn = create_load_balancer(sg_id, subnet_ids)

    # Step 8: Create a target group in the same VPC
    target_group_arn = create_target_group(vpc_id)

    # Step 9: Register EC2 instances as targets
    register_targets(target_group_arn, instance_ids)

    # Step 10: Create a listener to forward HTTP requests
    create_listener(lb_arn, target_group_arn)

if __name__ == "__main__":
    main()
```

- `elbv2.create_load_balancer()`: Creates a new Network Load Balancer, which distributes traffic across multiple targets (EC2 instances and IP addresses).
- `elbv2.create_target_group()`: Creates a target group, which is a logical grouping of EC2 instances or IP addresses to which the load balancer directs traffic.
- `ec2.create_security_group()`: Creates a security group that acts as a virtual firewall for the instances, controlling inbound and outbound traffic.
- `ec2.authorize_security_group_ingress()`: Adds inbound rules to the security group, allowing traffic for SSH (port 22) and HTTP (port 80) from all IP addresses (0.0.0.0/0).
- `ec2.describe_availability_zones()`: Retrieves a list of available availability zones in the region, ensuring that instances are created in different zones for fault tolerance.
- `ec2.create_key_pair()`: Creates a key pair for SSH access to the EC2 instances. The private key is saved locally, and the public key is associated with the instances.
- `ec2.run_instances()`: Launches EC2 instances in different availability zones, assigning them to the previously created security group and key pair.
- `elbv2.register_targets()`: Registers the EC2 instances to the target group so that the load balancer can send traffic to them.
- `elbv2.create_listener()`: Creates a listener for the load balancer that listens for HTTP traffic on port 80 and forwards the requests to the target group.

Results

Created a security group, authorised inbound traffic (SSH and HTTP), retrieved availability zones, created a key pair for SSH access, launched two EC2 instances with ID `i-0d275177fc9f9ac28` and `i-04cdb45522154b089`, retrieved available subnets, created an ALB with the retrieved subnets, created a target group, registered the EC2 instances with the target group, and set up a listener on the ALB to forward HTTP traffic.

[3] Test the Application Load Balancer

3.1 SSH to each of the two instances

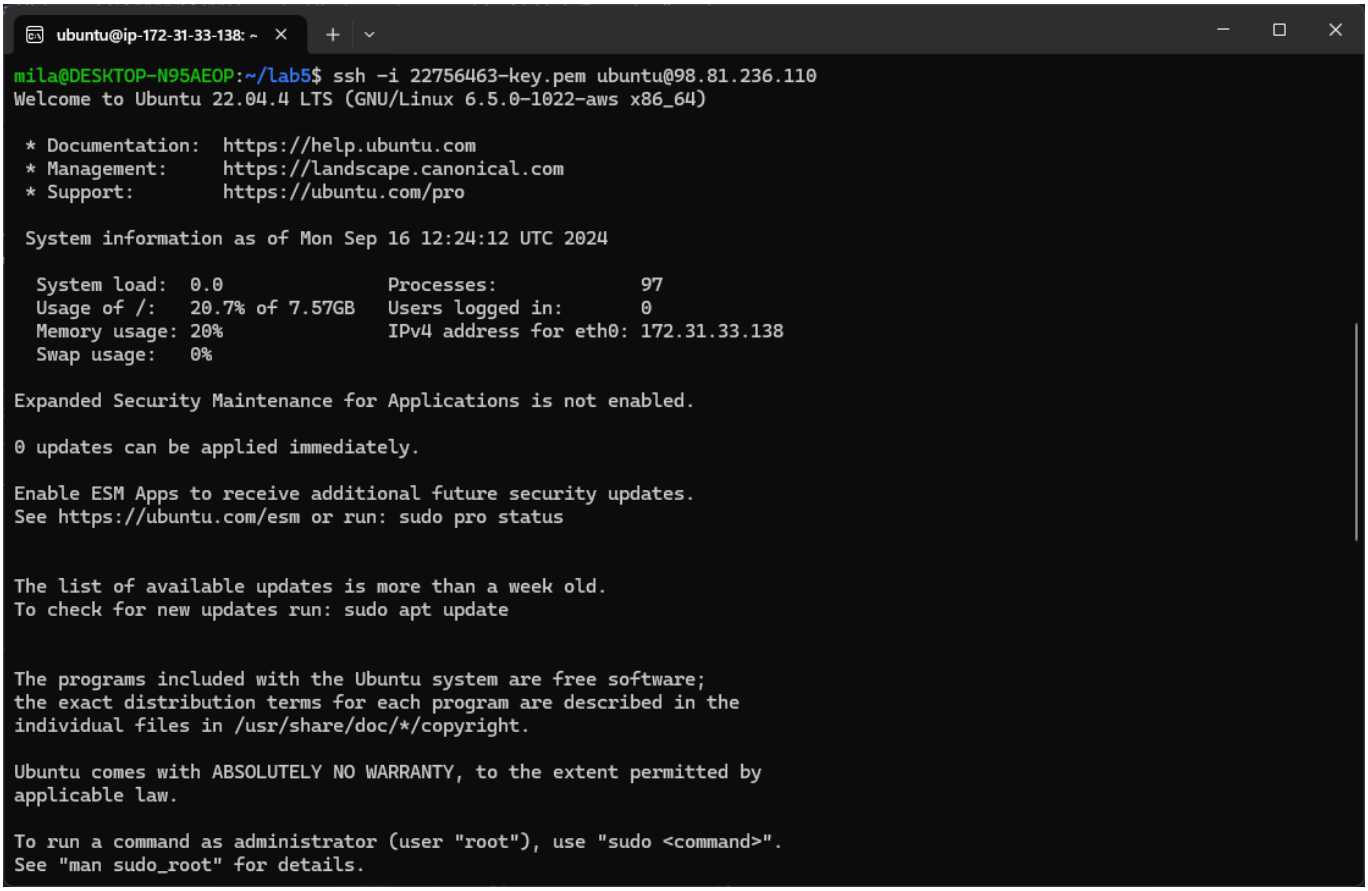
I obtained the public IP address of each EC2 instances from the AWS console. They are `98.81.236.110` and `34.204.0.107`. Then I ssh to each of the two instances using commands `ssh -i 22756463-key.pem ubuntu@98.81.236.110` and `ssh -i 22756463-key.pem ubuntu@34.204.0.107`.

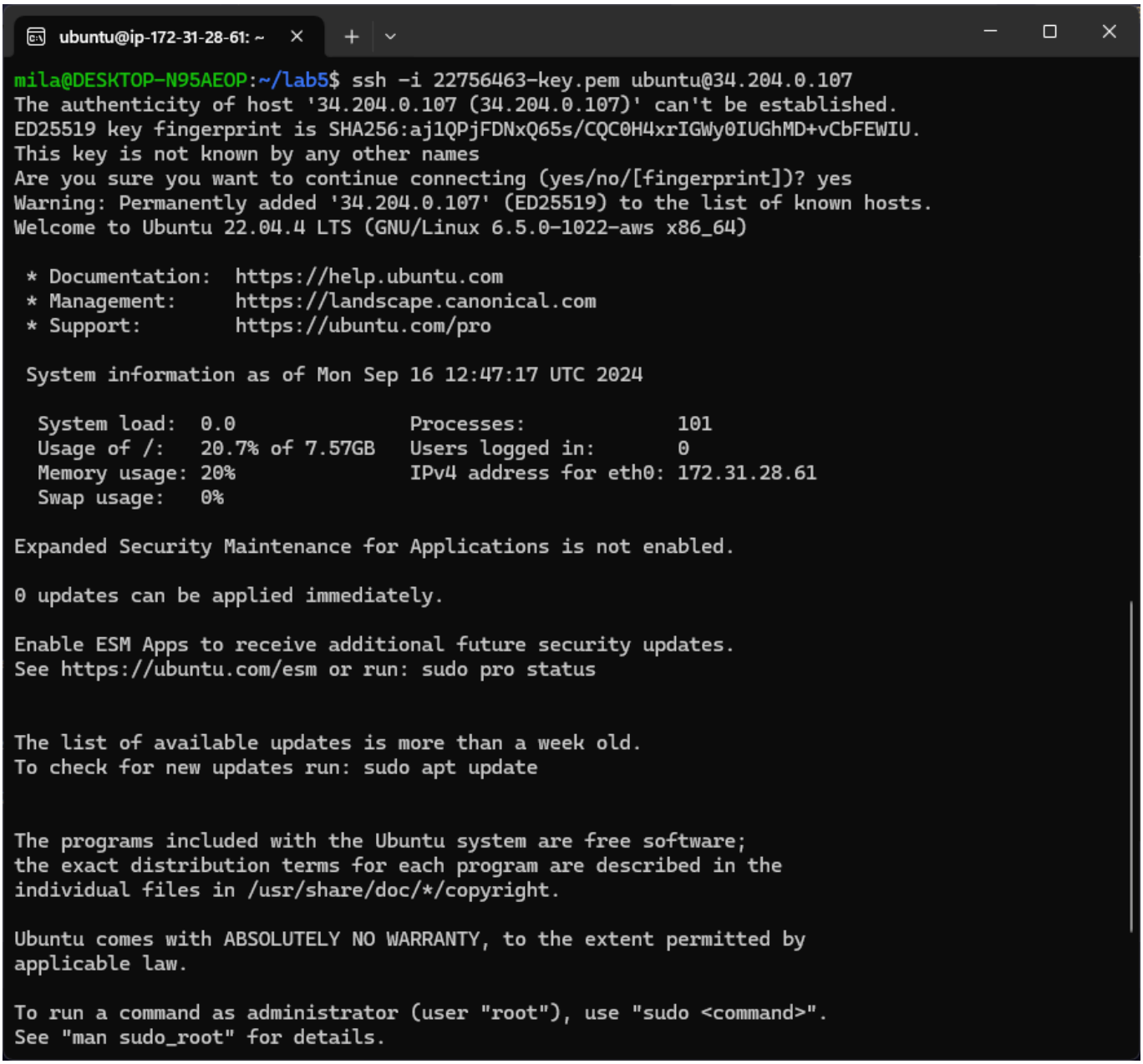
Commands

- `ssh`: This is the command to initiate an SSH (Secure Shell) connection, which allows you to remotely connect to another machine securely over the internet.
- `-i`: The -i option specifies the identity file (in this case, 22756463-key.pem). This is the private key file that corresponds to the public key stored on my EC2 instance.
- `ubuntu`: This is the username for my EC2 instance. Since my instance is running Ubuntu, the default username is ubuntu.

Results

I sshed to EC2 instances:





3.2 Update each instance

I updated each instance using command `sudo apt-get update`. After running this command, the system is prepared to install or upgrade packages based on the latest information from the repositories.

Commands

- `sudo`: Run the command with elevated (root) permissions. Installing software or making changes to system-wide configurations requires superuser privileges, so sudo is necessary for this command.
- `apt-get`: It is the Advanced Package Tool command-line interface for managing packages on Linux. It performs low-level operations, including installing, upgrading, and removing software packages, as well as updating the local package index. Note: `apt` (without `-get`) is a slightly newer version of the command with more user-friendly output but performs the same operations.
- `update`: The update option instructs apt-get to update the package index. It retrieves the latest list of available packages, versions, and their dependencies from the package repositories configured in the system.

`sudo apt-get update` updates the local list of available software packages and their versions from the repositories but does not install or upgrade any software.

Results

Updated each instance:

```
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates InRelease [128
kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports InRelease
[127 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 Packages
[14.1 MB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe Translation-en
[5652 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 c-n-f
Metadata [286 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64
Packages [217 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse Translation-
en [112 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 c-n-f
Metadata [8372 B]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64
Packages [2023 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main
Translation-en [352 kB]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-
n-f Metadata [17.8 kB]
Get:14 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted
amd64 Packages [2437 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted
Translation-en [419 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted
amd64 c-n-f Metadata [616 B]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64
Packages [1124 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe
Translation-en [261 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64
c-n-f Metadata [26.1 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse
amd64 Packages [43.3 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse
Translation-en [10.8 kB]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse
amd64 c-n-f Metadata [444 B]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64
Packages [67.8 kB]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main
```

```
Translation-en [11.1 kB]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64
c-n-f Metadata [388 B]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/restricted
amd64 c-n-f Metadata [116 B]
Get:27 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe
amd64 Packages [28.8 kB]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe
Translation-en [16.5 kB]
Get:29 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe
amd64 c-n-f Metadata [672 B]
Get:30 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/multiverse
amd64 c-n-f Metadata [116 B]
Get:31 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1806
kB]
Get:32 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [295
kB]
Get:33 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata
[13.3 kB]
Get:34 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages
[2377 kB]
Get:35 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en
[409 kB]
Get:36 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 c-n-f
Metadata [584 B]
Get:37 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[902 kB]
Get:38 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en
[176 kB]
Get:39 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f
Metadata [19.2 kB]
Get:40 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages
[37.2 kB]
Get:41 http://security.ubuntu.com/ubuntu jammy-security/multiverse Translation-en
[7588 B]
Get:42 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 c-n-f
Metadata [228 B]
Fetched 33.6 MB in 6s (5438 kB/s)
Reading package lists... Done
```

3.3 Install apache2 in each instance

I installed apache2 by using command `sudo apt install apache2`.

Commands

- `sudo`: Run the command with elevated (root) permissions. Installing software or making changes to system-wide configurations requires superuser privileges, so sudo is necessary for this command.
- `apt`: The Advanced Package Tool, which is the package manager for Ubuntu. It retrieves the software packages from the distribution's software repositories.
- `install`: The operation to install the specified package.

- `apache2`: This is the name of the package I want to install. Apache 2 is a popular web server software. Apache 2 is used to host websites and serve webpages over the HTTP/HTTPS protocols.

Results

Installed apache2:

```
Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service → /lib/systemd/system/apache2.service.
Created symlink /etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service → /lib/systemd/system/apache-htcacheclean.service.
Processing triggers for ufw (0.36.1-4ubuntu0.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.8) ...
Scanning processes...
Scanning linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

3.4 Edit HTML tags

I edited the `<title>` and `</title>` tags by using command `sudo nano /var/www/html/index.html`.

Commands

- `sudo`: Run the command with elevated (root) permissions.
- `nano`: Open, edit and save text tiles directly from the terminal.
- `/var/www/html/index.html`: The file path to the `index.html` file I want to edit. `/var/www/html/` is the default directory where Apache web server stores website files. The `index.html` file is typically the default web page served when someone accesses the website hosted on the server.

Results

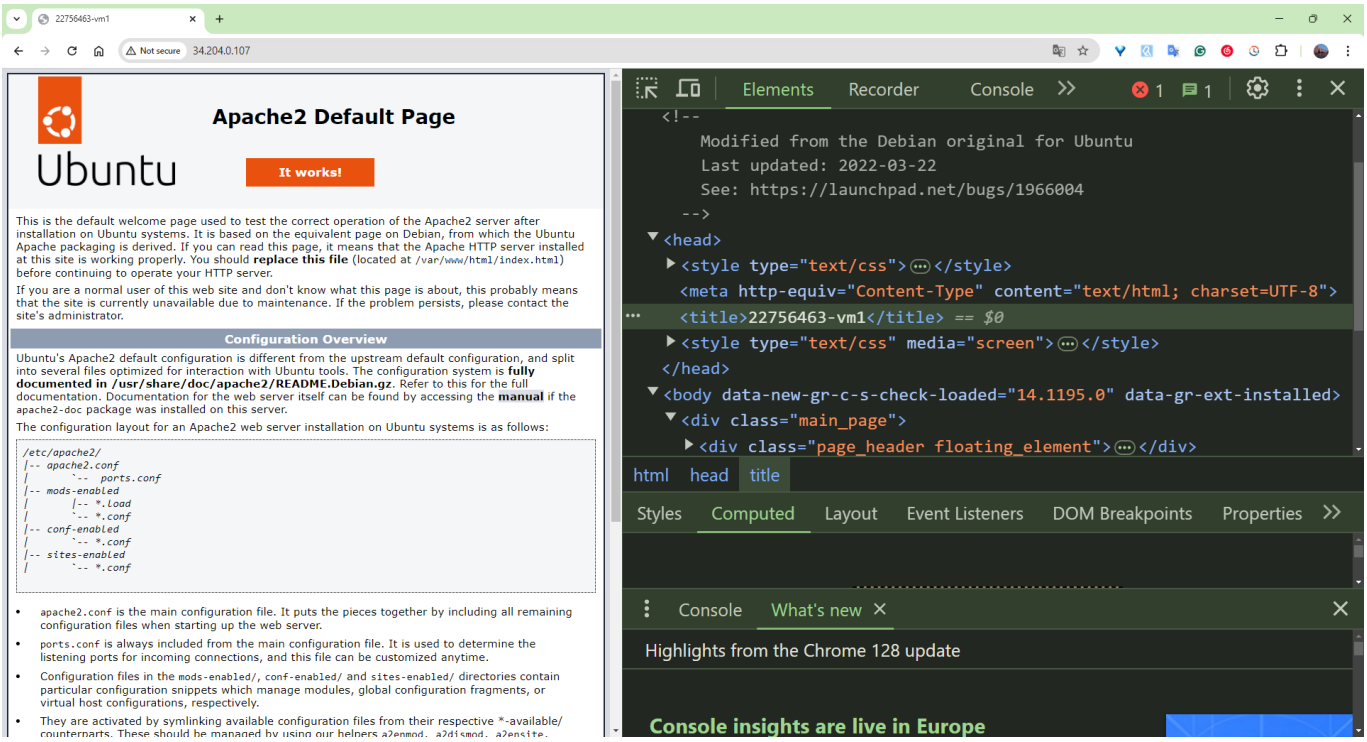
Replaced content between the `<title>` and `</title>` tags with the instance names.

3.5 Access instances by IP addresses

I accessed each instance by their IP addresses in the Chrome browser. Their instance names were shown as the webpage title in the top left corner. I also inspected the `<title>` tags to see the instance names.

Results

Instance name: 22756463-vm1, public IP address: 34.204.0.107.



Instance name: 22756463-vm2, public IP address: 98.81.236.110.

